Ranking Queries with Confidence Guarantees over Quantum Circuit-Cutting Reconstructions (Extended Version)

Jiachen Shen
University of Houston
Houston, TX, USA
jshen28@cougarnet.uh.edu

Hui Zhong
Miami University
Oxford, OH, USA
zhongh7@miamioh.edu

***Abstract*—Circuit cutting runs a quantum circuit that is too large for a device as many smaller subexperiments. It stores the output distribution over all bitstrings in factorized form, as a sum of products of small tables with signs, and each table is estimated from a finite number of measurements. Cutting systems answer top-k queries, which ask for the k most likely strings, on this representation without materializing the distribution. However, they rank the estimate, so a returned string can be wrong, and the answer does not say which strings are wrong. We answer top-k and threshold queries over the factorized representation with a confidence guarantee. Each string is labelled in, out or undecided. If the shots are independent and every term of the cut expansion is run, then with a probability chosen by the user, all in and out labels are correct at the same time. The guarantee covers every string, including the strings that the query never evaluates. We bound the measurement distributions of all subexperiments at once and push these bounds through the sum-product structure. We then tighten them near the boundary of the answer. A branch-and-bound search returns the same labels as full enumeration, and the guarantee still holds when more shots are added in fresh batches. On held-out circuits of the quantum approximate optimization algorithm (QAOA) with equal numbers of shots, our method labels on average 74% of the strings that are surely among the ten most likely as in, while per-category Clopper-Pearson bounds label 2% and a uniform Hoeffding-type bound labels none.**

***Index Terms*—top-k queries, uncertain data, factorized representations, confidence bounds, quantum circuit cutting**

I. Introduction

Circuit cutting stores the output distribution of a large quantum circuit in factorized form, as a sum of products of small tables with signs, and each table is estimated from a finite number of measurements. Like the query processor of a probabilistic database, a cutting system answers ranking queries directly on this uncertain representation, without materializing the distribution. However, current systems return the answer for the estimate, and they do not say which of the returned strings are correct.

A quantum circuit acts on qubits, the quantum counterpart of bits. Each run of the circuit is called a shot. It measures m output qubits and returns a bitstring of m bits, so the circuit defines a probability distribution over 2^m strings. Applications ask which strings are the most likely, which is a top-k query, or which strings have a probability above a threshold. For example, the strings of largest probability are the candidate solutions that an optimization circuit returns. When a circuit needs more qubits than a device holds, it can be run by cutting it [1]–[3]. Cutting splits the circuit into subcircuits that fit the device, and it replaces every cut by a quasi-probability decomposition, which is a sum of operations with signs that each subcircuit can run alone. Every term of the expansion fixes a small circuit, called a subexperiment, and the device runs each subexperiment for a finite number of shots. The reconstruction then combines the measured frequencies of the subexperiments into the factorized form. For example, a circuit with 40 output qubits that is cut into two halves stores tables of 2^{20} rows per term, while the distribution it represents has 2^{40} rows (Section II-A).

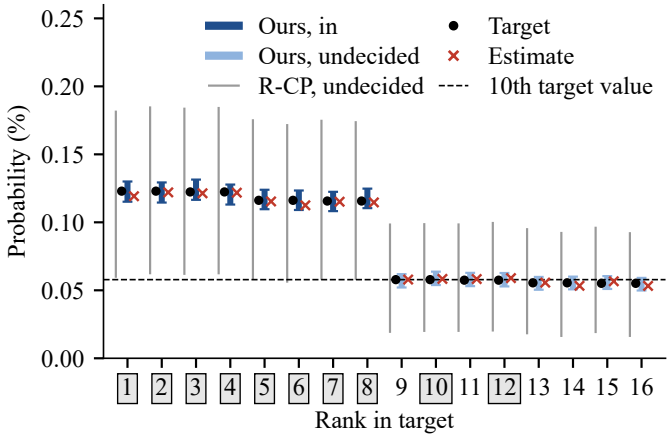


Fig. 1. Top-10 query on a 24-qubit QAOA circuit: intervals of our method and of per-category Clopper-Pearson bounds (R-CP). Boxes mark the top 10 of the estimate.

Current cutting systems already answer ranking queries without materializing the distribution, but their answers rest on the estimate. QCut [4], an open-source cutting library, keeps the reconstruction as a product over subcircuits and returns the k largest values of the estimate by branch and bound. Its answer consists of the most likely strings of the

estimate [4]. However, the factor tables hold shot frequencies multiplied by signs, so the estimate can be negative, and its ranking can differ from the ranking of the quantity it estimates. Fig. 1 shows an example from our benchmark. A depth-2 circuit of the quantum approximate optimization algorithm (QAOA) with $m = 24$ output qubits is cut by two gate cuts into two subcircuits, which gives 50 subexperiments, and each subexperiment runs $N = 10^5$ times. For this circuit, a simulator computes the target distribution to numerical precision (Section VI-A). One of the ten strings that `top()` returns on the estimate is not among the ten most likely strings of the target. This string is ranked twelfth in the target, and its estimated probability of 0.059% places it ninth in the estimate, while its probability of 0.057% is below the tenth largest value of 0.058%. Because the returned values carry no interval, the answer does not mark the wrong string. With our bounds, the eight strings that are clearly separated from the rest are labelled IN, the 39 strings near the tenth value stay undecided, and all other strings are labelled OUT. In contrast, the per-category Clopper-Pearson bounds (R-CP in the figure) are so wide that they label no string IN.

Two lines of work come close to this problem. Neumann et al. [5] bound the sampling error of the reconstructed probabilities of a cut circuit with Hoeffding's inequality and a union bound. However, a bound on values gives a ranking answer only together with a decision rule and a search. Section VI measures how many strings a uniform bound of this kind decides when it is given both. In databases, top-k queries over uncertain data are well studied [6]–[8]. For example, Dylla et al. [9] prune the candidates of a top-k query over non-materialized views with lower and upper probability bounds, which is close to the search that our problem needs. Their probabilities, however, belong to a known probabilistic database. In our setting, the distribution is unknown and can only be reached through factors with signs that are estimated from shots. The confidence of an answer is therefore its coverage over the random shots.

We study top-k and threshold queries with a confidence guarantee over this representation (Section II-C). Given a level δ, the answer labels each of the 2^m strings as guaranteed in (IN), guaranteed out (OUT) or undecided (UND). With probability at least $1-\delta$, every IN label and every OUT label is correct, jointly for all strings. An UND label makes no claim. It marks a string that the shots do not separate from the boundary of the answer, which is the k-th largest value or the threshold. The guarantee concerns the reconstruction at the distributions that the subexperiments produce, under exact expansion of the cuts and independent shots (Section II-B).

The key idea is to bound the raw measurement distributions of all subexperiments at once and to push these bounds through the sum-product structure, so that one event guarantees every answer, including the answers for strings that the search never visits. Applied to all completions of a partial bitstring, the same bounds bound a whole branch of the search. A branch whose bound lies below the boundary is therefore labelled OUT as a whole, on the same event, without evaluating its strings. For the few strings near the boundary, a small pilot run selects scores and score differences, and Bernstein's inequality [10] then tightens their bounds. When the user asks for more shots, the new shots come in fresh batches and go to the subexperiments that decide the boundary.

Contributions.

- *Bounds for all strings from one event* (Section III). Confidence regions on the raw measurement categories bound all 2^m strings at once, and Bernstein bounds tighten them at the boundary.
- *Three-valued answers without materialization* (Section IV). The IN and OUT labels of our top-k and threshold rules are all correct on that event, and a branch-and-bound search returns the same labels as full enumeration.
- *Appended shots driven by the query* (Section V). Rounds with fresh batches keep the level δ under any data-dependent allocation and stopping rule.
- *Evaluation* (Section VI). No label in 10,831 runs is wrong. On held-out QAOA circuits with equal total shots, our method labels on average 74% of the sure top-10 members IN, against 2% for per-category Clopper-Pearson bounds.

II. Preliminaries and Problem Definition

This section first defines the stored representation and the sampling model that produces its tables. It then defines the two ranking queries and their three-valued answers.

A. Cut Circuits as a Factorized Representation

We query the joint measurement outcome of m qubits of a quantum circuit, which we call the output qubits. Measuring the output qubits yields a bitstring $x \in \{0,1\}^m$, so the queried distribution has 2^m entries. Circuit cutting splits the circuit into s smaller subcircuits, and a small device can run each of them on its own [1]–[3]. The output qubits are divided among the subcircuits in the same way. Accordingly, we write $x = (x_1, \ldots, x_s)$, where the block $x_i \in \{0,1\}^{m_i}$ holds the m_i output bits measured on subcircuit i. A subcircuit without output bits has $m_i = 0$, and its block x_i is the empty string.

Each cut is replaced by a quasi-probability decomposition, which writes the cut operation as a signed sum of operations that each subcircuit can execute alone [1], [2]. Expanding all cuts gives a finite set $\mathcal{G}$ of groups. Group g has a real coefficient c_g, and on every subcircuit i it runs one subexperiment $e(i,g)$ from a finite set $\mathcal{S}$ of distinct circuits. Two groups share a subexperiment whenever they run the same circuit. Within one group, the s subcircuits use s distinct subexperiments.

A single run, or shot, of subexperiment e returns a raw category $z \in Z_e$, which records all classical bits of the run, namely the output bits of the subcircuit and the bits measured at its cuts. The category set Z_e is therefore fixed by the circuit, and it contains every possible pattern of these bits, whether or not the pattern is observed. We write q_e for the unknown distribution of z on Z_e.

The factor of group g on subcircuit i assigns to every block value $y_i \in \{0,1\}^{m_i}$ the signed linear functional

$$u_{i,g}(y_i) = \sum_{z \in Z_e} a_{i,g,y_i}(z)\, q_e(z), \qquad e = e(i,g). \tag{1}$$

Here $a_{i,g,y_i}(z) = \sigma_{i,g}(z)$ if the output bits of z equal y_i and $a_{i,g,y_i}(z) = 0$ otherwise. The sign $\sigma_{i,g}(z) \in \{-1,+1\}$ is the product of the signs of the cut measurements in z that group g uses on subcircuit i. A factor can therefore be negative, and its values need not sum to one. For a subcircuit with $m_i = 0$, the factor is a single number. The reconstruction first forms the signed table

$$T(x) = \sum_{g \in \mathcal{G}} c_g \prod_{i=1}^{s} u_{i,g}(x_i) \tag{2}$$

and then sets

$$p(x) = \frac{1 + \sum_{y \in \{0,1\}^m} T(y)}{2^m} - T(x). \tag{3}$$

Because the table is separable, the constant in (3) is cheap to compute. Specifically, $\sum_y T(y) = \sum_g c_g \prod_i S_{i,g}$, where the mass $S_{i,g} = \sum_{y_i} u_{i,g}(y_i) = \sum_z \sigma_{i,g}(z)\, q_e(z)$ is itself a signed linear functional of q_e. With the global sign $(-1)^{w+1}$ of the open-source library QCut [4] folded into c_g, where w is the number of wire cuts, (2) and (3) give exactly the reconstruction that QCut computes, up to its indexing of the table by the bitwise complement of x.

Equations (1) to (3) define a factorized representation of a distribution in the database sense. In this representation, the 2^m values of p are never stored. Instead, the system stores $|\mathcal{G}|$ products, each of s small signed tables, and the table of group g on subcircuit i has 2^{m_i} rows. For example, take a circuit with $m = 40$ output qubits that is cut into two subcircuits with 20 output qubits each. Every group then stores two tables of 2^{20} rows, while the distribution it represents has 2^{40} rows. Unlike the tables of a probabilistic database, these tables are signed, and each of them is a linear image of a distribution q_e that is known only through samples.

For any collection $r = (r_e)_{e \in \mathcal{S}}$ of distributions on the category sets, we write $p_r(x)$ for the value of (3) when every q_e is replaced by r_e. The *target* of every query is $p = p_q$, the reconstruction evaluated at the true distributions. When the subexperiments run without noise, p equals the output distribution of the uncut circuit on the output qubits, because the decomposition is exact [1], [2]. When they run on a noisy device, p is the same formula evaluated at the distributions that the device produces, and it can then take negative values. For this reason, no result in this paper relies on $p \geq 0$. On the other hand, the identity $\sum_x p_r(x) = 1$ follows from (3) for every r, so it holds whatever the counts are.

B. Sampling Model

Subexperiment e is run N_e times. We write $n_{e,z}$ for the number of shots that returned category z and $\hat{q}_e(z) = n_{e,z}/N_e$ for the empirical distribution. Substituting the empirical distributions into the reconstruction gives the plug-in estimate $\hat{p} = p_{\hat{q}}$ of the target. All guarantees in this paper hold under the following model.

Assumption 1 (Sampling model)**.** (A1) The expansion is exact. All groups in $\mathcal{G}$ are evaluated, and the cuts, the groups, the coefficients c_g, the subexperiments and the category sets Z_e are fixed in advance. (A2) The shots of subexperiment e are independent draws from q_e, and different subexperiments draw independently of each other. (A3) The shot counts N_e are fixed before the shots used for inference are observed, either by design or by a pilot run on separate shots.

On a simulator, (A2) holds by construction. On a device, (A2) requires that the device produces one fixed distribution for every subexperiment while the shots are taken. For (A1), QCut's `exact` expansion mode evaluates every group [4].

C. Queries and Three-Valued Answers

We study two ranking queries over the target p. Given a threshold $\tau \in \mathbb{R}$, the *threshold query* asks which strings satisfy $p(x) > \tau$. Given an integer k with $1 \leq k \leq 2^m$, the *top-k query* asks for the k strings with the largest values of p. Because the values of p can be tied, we define the top-k query through sets. A *top-k set* of p is any set $\mathcal{T}$ of k strings such that $p(y) \geq p(z)$ for all $y \in \mathcal{T}$ and $z \notin \mathcal{T}$. A string x is a *sure member* if it lies in every top-k set, which holds exactly when $|\{y \neq x : p(y) \geq p(x)\}| \leq k-1$. It is a *sure non-member* if it lies in no top-k set, which holds exactly when $|\{y : p(y) > p(x)\}| \geq k$.

An answer assigns one of three labels to every string. For the threshold query, the label IN is correct for x if $p(x) > \tau$, and the label OUT is correct if $p(x) < \tau$. For the top-k query, IN is correct for a sure member, and OUT is correct for a sure non-member. The label UND (undecided) makes no claim, so it is always correct. These definitions break no tie arbitrarily. For example, a string with $p(x) = \tau$ has UND as its only correct label, and so does a string whose value is shared by the strings at positions k and $k+1$ of the sorted order.

Definition 1 (Ranking query with a confidence guarantee)**.** The input is the cut representation of Section II-A, the counts $n_{e,z}$ of all subexperiments, a level $\delta \in (0,1)$, and a query, which is either a top-k query or a threshold query. The output assigns a label in $\{\mathsf{IN}, \mathsf{OUT}, \mathsf{UND}\}$ to each of the 2^m strings. It is returned as an explicit list of strings with their labels, together with one label shared by all strings outside the list. The output must satisfy

$$\Pr[\text{every } \mathsf{IN} \text{ label and every } \mathsf{OUT} \text{ label is correct}] \geq 1 - \delta,$$

where the probability is over the shots and the event covers all 2^m strings.

The event in Definition 1 covers every string, including the strings that an algorithm never evaluates. The guarantee is stated per query, so each query receives its own level δ. Within this requirement, a useful procedure labels most strings IN or OUT at a given number of shots, while it evaluates only a few of the 2^m strings.

TABLE I
QUANTITIES OF THE MODEL, THEIR COUNTERPART IN QUERY PROCESSING, AND THEIR SIZES IN THE BENCHMARK (ARITHMETIC AND QAOA CIRCUITS).

Model	Counterpart	Benchmark
string x	row of the result	$m = 16$ to 40
factor table $u_{i,g}$	stored relation	2^{m_i} rows, $m_i \le 20$
group g, sign c_g	term of a signed sum	$\lvert\mathcal{G}\rvert = 36$ or 64
subexperiment e	sampled source	$\lvert\mathcal{S}\rvert = 48$ or 50
shot	one sample of a source	$N = 10^3$ to 10^7
$p(x)$	score of a row	ranked by the query
k, τ, δ	query parameters	$k \le 10$, $\delta = 0.05$

Example 1. The circuit of Fig. 1 has $m = 24$ output qubits. Two gate cuts split it into $s = 2$ subcircuits with 12 output bits each. The expansion has $\lvert\mathcal{G}\rvert = 36$ groups, which run $\lvert\mathcal{S}\rvert = 50$ distinct subexperiments with 4,096 or 16,384 raw categories each. The system therefore stores 36 products of two tables with 2^{12} rows each, while p has $2^{24} \approx 1.7 \cdot 10^7$ values. For the top-10 query, the strings ranked first to tenth in Fig. 1 are the sure members, and every other string is a sure non-member. In the figure, the answer of our method labels the first eight strings IN, 39 strings UND and all other strings OUT, so all its IN and OUT labels are correct. Table I relates the quantities of the model to query processing and lists their sizes in our benchmark.

III. CONFIDENCE REGIONS AND BOUND PROPAGATION

The guarantee of this section rests on one event Ω of probability at least $1 - \delta$. On Ω, the true distributions of all subexperiments lie in a region computed from the counts, and the target value of every string lies between a computed lower bound $L(x)$ and a computed upper bound $U(x)$. To this end, the section first builds the region (Section III-A), then propagates it to all 2^m strings (Section III-B), and finally refines the bounds of the few strings that decide the query (Section III-C). The error level δ is split over the region, the selected scores and the selected pairs, and, with the tighter terms, also over the factor functionals. The full proofs of all theorems are in Appendix A.

A. A Region over Raw Categories

We construct intervals for all categories of every subexperiment at once. For $N \ge 1$ trials with n successes and a level $\alpha \in (0, 1)$, the two-sided Clopper-Pearson interval [12] has the lower endpoint 0 if $n = 0$ and the $\alpha/2$ quantile of the Beta$(n, N - n + 1)$ distribution otherwise. Its upper endpoint is 1 if $n = N$ and the $1 - \alpha/2$ quantile of the Beta$(n + 1, N - n)$ distribution otherwise. For every $e \in \mathcal{S}$ and $z \in Z_e$, let $[\underline{q}_{e,z}, \overline{q}_{e,z}]$ be this interval for $n = n_{e,z}$ and $N = N_e$ at the level

$$\alpha_e = \frac{\delta_0}{\lvert\mathcal{S}\rvert \, \lvert Z_e\rvert}. \tag{4}$$

The region of subexperiment e intersects these intervals with the probability simplex $\Delta(Z_e)$:

$$C_e^0 = \{r_e \in \Delta(Z_e) : \underline{q}_{e,z} \le r_e(z) \le \overline{q}_{e,z} \text{ for all } z \in Z_e\}, \tag{5}$$

and the joint region is the product $C^0 = \prod_{e \in \mathcal{S}} C_e^0$.

Theorem 1 (Region)**.** *Under Assumption 1, for all shot counts $N_e \ge 1$ and all distributions q_e, $\Pr[q \in C^0] \ge 1 - \delta_0$. Moreover, let $a \in \{-1, 0, +1\}^{Z_e}$, let $\rho = 1 - \sum_z \underline{q}_{e,z}$ and $v_0 = \sum_z a_z \underline{q}_{e,z}$, and let $W_t = \sum_{z : a_z = t} (\overline{q}_{e,z} - \underline{q}_{e,z})$ for $t \in \{-1, 0, +1\}$. Then*

$$\min_{r_e \in C_e^0} a^\top r_e = v_0 - \min(\rho, W_{-1}) + (\rho - W_{-1} - W_0)_+,$$
$$\max_{r_e \in C_e^0} a^\top r_e = v_0 + \min(\rho, W_{+1}) - (\rho - W_{+1} - W_0)_+.$$

Proof sketch. Under (A2) and (A3), conditionally on the pilot of Section III-C when one is used, each count $n_{e,z}$ is binomial with N_e trials and success probability $q_e(z)$. The Clopper-Pearson interval misses its parameter with probability at most α_e, because the event that the lower endpoint exceeds the parameter is an upper tail of the binomial distribution of mass at most $\alpha_e/2$, and symmetrically for the upper endpoint. The endpoint conventions cover the parameters 0 and 1. A union bound over all categories of the fixed sets Z_e, observed or not, and over all subexperiments gives a failure probability of at most $\sum_e \lvert Z_e\rvert \, \alpha_e = \delta_0$. The simplex constraint adds no failure event, and the union bound needs no independence. For the extrema, write $r_e(z) = \underline{q}_{e,z} + d_z$ with $0 \le d_z \le \overline{q}_{e,z} - \underline{q}_{e,z}$ and $\sum_z d_z = \rho$. Grouping the free mass by coefficient turns the problem into three capacities W_{-1}, W_0, W_{+1} that share the mass ρ. The minimum fills the coefficients -1, 0 and $+1$ in this order, and the maximum fills them in the reverse order. The region is nonempty because every interval contains the empirical proportion. □

A category that was never observed has the interval $[0, 1 - (\alpha_e/2)^{1/N_e}]$, and its upper endpoint is at most $\ln(2/\alpha_e)/N_e$. For example, on the circuit of Example 1 with $\delta_0 = 0.005$, $N_e = 10^5$ and $\lvert Z_e\rvert = 16{,}384$, this upper endpoint is at most $2.0 \cdot 10^{-4}$. The region therefore covers every category of Z_e, including the many categories that a finite number of shots never reaches. All unseen categories share one interval. For a factor of the form (1), the number of unseen categories with each coefficient follows from the bit pattern of a, so computing the extrema of Theorem 1 needs one pass over the observed categories. Applying Theorem 1 to (1) gives an interval $[\underline{u}_{i,g}(y_i), \overline{u}_{i,g}(y_i)]$ for every factor value and an interval $[\underline{S}_{i,g}, \overline{S}_{i,g}]$ for every mass. Each interval is the exact range of its functional over C_e^0. For example, a mass has the coefficient $\sigma_{i,g}(z) \in \{-1, +1\}$ on every category, so $W_0 = 0$, and its minimum places the free mass ρ first on the categories with coefficient -1.

B. Propagation to All Strings

Interval arithmetic on (2) and (3) turns the factor intervals into bounds on the target value of every string. Let $L^0(x)$ and

$U^0(x)$ be the lower and upper bound that interval arithmetic gives for (3). Specifically, every factor and mass is replaced by its interval, and the product of two intervals is the interval spanned by the four products of their endpoints. A sum of intervals adds the corresponding endpoints, and a coefficient multiplies an interval with its sign taken into account.

Lemma 1 (Propagation)**.** *For every $r \in C^0$ and every $x \in \{0,1\}^m$, $L^0(x) \le p_r(x) \le U^0(x)$.*

Proof sketch. For $r \in C^0$, every factor value and every mass lies in its interval by Theorem 1. Each interval operation contains the result of the same operation on any members of its operands. Induction over the expression (3) proves the claim. For example, the product of the intervals $[-1/4, 1/2]$ and $[1/2, 1]$ is $[-1/4, 1/2]$, because the four endpoint products are $-1/8$, $-1/4$, $1/4$ and $1/2$. □

On the event $q \in C^0$, Lemma 1 bounds the target value of all 2^m strings at the same time. The bounds of one string cost $O(|\mathcal{G}|\, s)$ interval operations, and the constant term is shared by all strings and computed once. However, interval arithmetic bounds every occurrence of a factor separately, even when several groups read the same subexperiment, and it charges each factor its full range over the region. Both effects widen the bounds, and Section III-C reduces them for the strings that decide the query.

Example 2. For the most likely string of Fig. 1, the Clopper-Pearson region with $\delta_0 = 0.05$ and interval propagation give $[0.059\%, 0.182\%]$, an interval about as wide as the probability 0.123% itself.

C. Refined Bounds for Selected Scores and Pairs

Pilot. Before inference, a pilot spends a small set of separate shots on every subexperiment. From the pilot estimate, it fixes a set A of strings with the largest pilot scores. For a top-k query, it also fixes a set P of pairs of strings of A whose pilot ranks lie in a window around k, so both strings of every pair belong to A. The pilot further fixes the error shares and the inference shot counts N_e. The inference shots are fresh, so (A1) to (A3) hold conditionally on the pilot with A, P and the N_e fixed. Every statement below holds conditionally on the pilot and therefore also unconditionally.

Selected functionals. Let F contain the score $p_r(x)$ for every $x \in A$ and the difference $p_r(x) - p_r(y)$ for every pair $\{x, y\} \in P$. Each $f \in F$ can be written as an affine combination of products of signed linear functionals,

$$f(r) = c_{0,f} + \sum_{h \in \mathcal{H}_f} c_{h,f} \prod_{i \in I_h} v_{h,i}(r), \tag{6}$$

where $\mathcal{H}_f$ is a finite set of products, I_h is the set of positions in product h, and each $v_{h,i}(r) = \sum_z a_{h,i}(z)\, r_{e(h,i)}(z)$ is a factor value or a mass from Section II-A and $|a_{h,i}(z)| \le 1$. For a score, $c_{0,f} = 2^{-m}$, and every group contributes one product of factor values with coefficient $-c_g$ and one product of masses with coefficient $c_g/2^m$. For a difference, the constant and the mass products cancel, and only the products of factor values of the two strings remain. No subexperiment occurs twice in one product.

Linear term. Moving r to $\hat{q}$ changes f to first order by $Z_f(r) = \sum_e \sum_z g_{e,f}(r, z)\,(\hat{q}_e(z) - r_e(z))$, where the weight of category z of subexperiment e is

$$g_{e,f}(r, z) = \sum_{(h,i):e(h,i)=e} c_{h,f}\, a_{h,i}(z) \prod_{j \in I_h,\, j \ne i} v_{h,j}(r). \tag{7}$$

The weight adds the contributions of all products that read e before any moment is taken. At $r = q$, the term $Z_f(q)$ is a sum of independent centered variables, one per shot. Shot ℓ of subexperiment e contributes $(g_{e,f}(q, Z_{e,\ell}) - \mathbb{E}_{q_e} g_{e,f}(q, Z))/N_e$, where $Z_{e,\ell}$ is its category and Z is a category drawn from q_e. Bernstein's inequality [10] bounds such a sum through its variance and its range,

$$V_f(q) = \sum_e \frac{\mathrm{Var}_{q_e}[g_{e,f}(q, Z)]}{N_e},$$

$$b_f(q) = \max_e \frac{\max_z g_{e,f}(q, z) - \min_z g_{e,f}(q, z)}{N_e}.$$

The variance is small when the weight is nearly constant on the categories that carry most of the mass of q_e, and a bound based on the variance benefits from this. In contrast, a bound that uses only the range, such as Hoeffding's inequality [13], still charges the full range of the weight in this case.

Computable radius. The quantities $V_f(q)$ and $b_f(q)$ depend on the unknown q, so we bound them over C^0. For every functional in (6), let $\hat{v}_{h,i} = v_{h,i}(\hat{q})$, let $[\underline{v}_{h,i}, \overline{v}_{h,i}]$ be its exact range over C^0 from Theorem 1, and let

$$r_{h,i} = \max\{\hat{v}_{h,i} - \underline{v}_{h,i},\ \overline{v}_{h,i} - \hat{v}_{h,i}\}, \qquad M_{h,i} = |\hat{v}_{h,i}| + r_{h,i}.$$

Let $\hat{g}_{e,f}(z) = g_{e,f}(\hat{q}, z)$ be the plug-in weight, and bound its deviation by

$$d_{e,f}(z) = \sum_{(h,i):e(h,i)=e} |c_{h,f}|\, |a_{h,i}(z)| \Big(\prod_{j \ne i} M_{h,j} - \prod_{j \ne i} |\hat{v}_{h,j}| \Big).$$

With $\mu_{e,f} = \sum_z \hat{g}_{e,f}(z)\, \hat{q}_e(z)$, let $H_{e,f}^2 = \sum_z \overline{q}_{e,z} (\hat{g}_{e,f}(z) - \mu_{e,f})^2$ and $J_{e,f}^2 = \sum_z \overline{q}_{e,z}\, d_{e,f}(z)^2$. The radius of f is

$$E_f = \sqrt{2 \overline{V}_f t_f} + \frac{\overline{b}_f t_f}{3} + R_f, \tag{8}$$

with the variance envelope, the range envelope and the remainder

$$\overline{V}_f = \sum_e \frac{(H_{e,f} + J_{e,f})^2}{N_e},$$

$$\overline{b}_f = \max_e \frac{\max_z (\hat{g}_{e,f} + d_{e,f})(z) - \min_z (\hat{g}_{e,f} - d_{e,f})(z)}{N_e},$$

$$R_f = \sum_{h \in \mathcal{H}_f} |c_{h,f}| \sum_{\mathcal{K} \subseteq I_h,\ |\mathcal{K}| \ge 2} \prod_{j \in \mathcal{K}} r_{h,j} \prod_{j \in I_h \setminus \mathcal{K}} M_{h,j},$$

and with $t_f = \ln(2|A|/\delta_s)$ for a score and $t_f = \ln(2|P|/\delta_p)$ for a difference. We write E_x for the radius of the score of x and E_{xy} for the radius of the difference of x and y.

Tighter terms. The following three changes shrink the radius and keep it valid on an event of the same kind. *(i)*

Factor radii. Let $\mathcal{F}$ be the set of distinct linear functionals (e, a) that occur in (6) for some $f \in F$, a set that the pilot fixes. For each of these functionals, $a \cdot (\hat{q}_e - q_e)$ is an average of N_e independent centered terms. Bernstein's inequality with $t_{\mathcal{F}} = \ln(2|\mathcal{F}|/\delta_{\mathcal{F}})$ therefore bounds it by $\rho_a = \sqrt{2\bar{\sigma}_a^2 t_{\mathcal{F}}/N_e} + (\max a - \min a)\, t_{\mathcal{F}}/(3N_e)$, where $\bar{\sigma}_a^2 = \sup_{r \in C_e^0} \sum_z r_z (a_z - a \cdot \hat{q}_e)^2$ is computed exactly by a greedy fill of the box and the simplex. Wherever the proof needs a deviation at the true q, we use $\min\{r_{h,i}, \rho_a\}$ in place of $r_{h,i}$. *(ii) Envelopes.* $H_{e,f}^2$ and $J_{e,f}^2$ use the supremum of the same second moments over C_e^0. This supremum is never larger than the bound with the upper endpoints $\overline{q}_{e,z}$. *(iii) Bilinear remainder.* When every product of f has two factors from disjoint sets of subexperiments, as for two subcircuits, the remainder is $\sum_h c_{h,f} d_{h,0} d_{h,1}$, where $d_{h,i}$ are the deviations of the factors. Given the shots of one side, it is again a sum of independent centered terms whose weights are bounded through (i), so Bernstein's inequality bounds it. We use the smaller of this bound and R_f. As a result, each score and each pair carries two events, so $t_f = \ln(4|A|/\delta_s)$ and $t_f = \ln(4|P|/\delta_p)$. The level splits as $\delta = \delta_0 + \delta_{\mathcal{F}} + \delta_s + \delta_p$, with $\delta_{\mathcal{F}} = 0$ when (i) is not used. In the experiments, $|A| = 128$, $\delta_0 = 0.005$, $\delta_{\mathcal{F}} = 0.01$, $\delta_s = 0.02$ and $\delta_p = 0.015$.

Theorem 2 (Refinement)**.** *Under Assumption 1 and conditionally on the pilot, let Ω be the event that $q \in C^0$ and $|f(\hat{q}) - f(q)| \le E_f$ for every $f \in F$. Then* $\Pr[\Omega] \ge 1 - \delta_0 - \delta_{\mathcal{F}} - \delta_s - \delta_p = 1 - \delta$.

Proof sketch. Write every factor at $\hat{q}$ as its value at q plus a deviation and expand each product in (6). The terms of degree one add up to $Z_f(q)$, and on $q \in C^0$ the terms of degree at least two add up to at most R_f. Bernstein's inequality [10] bounds $Z_f(q)$ through its variance $V_f(q)$ and range $b_f(q)$ with probability at least $1 - 2e^{-t_f}$. On $q \in C^0$, the weights at q lie within $d_{e,f}$ of the plug-in weights, so a variance taken about $\mu_{e,f}$ and the triangle inequality in $L^2(q_e)$ give $V_f(q) \le \overline{V}_f$ and $b_f(q) \le \bar{b}_f$. The tighter terms add the factor events and one conditional Bernstein event per functional for the bilinear remainder. A union bound over all events gives the level, and averaging over the pilot completes the proof. □

The refined bounds take the better of the two constructions on the selected strings:

$$L(x) = \begin{cases} \max\{L^0(x),\, \hat{p}(x) - E_x\}, & x \in A, \\ L^0(x), & x \notin A, \end{cases} \qquad (9)$$

$$U(x) = \begin{cases} \min\{U^0(x),\, \hat{p}(x) + E_x\}, & x \in A, \\ U^0(x), & x \notin A. \end{cases}$$

By Lemma 1 and Theorem 2, on Ω all 2^m strings satisfy $L(x) \le p(x) \le U(x)$, and every pair in P satisfies $|(\hat{p}(x) - \hat{p}(y)) - (p(x) - p(y))| \le E_{xy}$. A pair radius bounds the difference of two scores directly. The weights of both strings on a shared subexperiment are added in (7) before the variance is taken, so the part of the two scores that moves together cancels in E_{xy}. Two separate score intervals, in contrast, each pay for that part. For single strings, the refinement narrows the interval of Example 2, for the most likely string of Fig. 1, to $[0.115\%, 0.130\%]$.

IV. Query Answering

This section turns the bounds of Section III into labels. Section IV-A states the decision rules and proves that, on Ω, all the IN and OUT labels that they give are correct for every string (Theorem 3). Building on this, Section IV-B gives a branch-and-bound search. The search evaluates only the branches and strings that its bounds keep, yet it returns the same labels as applying the rules to all 2^m strings (Theorem 4).

A. Decision Rules

Threshold query. The rule compares the bounds (9) with the threshold τ. A string is labelled IN if $L(x) > \tau$, OUT if $U(x) < \tau$, and UND otherwise.

Top-k query. Besides the bounds, the rule also compares strings through the pairs in P. For $\{x, y\} \in P$, we say that x is *shown above* y, written $x \succ y$, if $\hat{p}(x) - \hat{p}(y) > E_{xy}$. We call this comparison the pair test of x and y. For every string x, let

$$\mathrm{Below}(x) = \{y \ne x : U(y) < L(x) \text{ or } x \succ y\},$$
$$\mathrm{Above}(x) = \{y \ne x : L(y) > U(x) \text{ or } y \succ x\}.$$

The strings in $\mathrm{Below}(x)$ are shown to score below x. The other $2^m - 1 - |\mathrm{Below}(x)|$ strings can therefore still score at least $p(x)$, and we call them the competitors of x. A string is labelled IN if at most $k-1$ competitors remain, that is, if $2^m - 1 - |\mathrm{Below}(x)| \le k-1$. It is labelled OUT if $|\mathrm{Above}(x)| \ge k$, that is, if at least k strings are shown to score above it, and we call these strings its witnesses. Otherwise it is labelled UND. If both tests hold, the label is OUT. In addition, if some refined interval is empty, that is, $L(x) > U(x)$ for some $x \in A$, every string is labelled UND. Neither of these two cases occurs on Ω.

Example 3. In Fig. 1, the interval and pair tests leave each of the eight most likely strings with at most nine competitors, so all eight are labelled IN. The ninth string keeps at least the eight strings above it as competitors. Its interval $[0.052\%, 0.062\%]$ also overlaps the intervals of the strings ranked tenth to sixteenth, and the pair tests do not separate it from enough of them, so it stays UND. Every string outside the 8 IN and 39 UND strings has at least ten witnesses and is therefore labelled OUT.

Theorem 3 (Soundness)**.** *Under Assumption 1, for a top-k query or a threshold query, the labels that the rules of Section IV-A assign to all 2^m strings satisfy*

$$\Pr[\textit{every IN and every OUT label is correct}] \ge 1 - \delta.$$

Proof sketch. We show that every IN and OUT label is correct on Ω, and Theorem 2 then gives the probability. On Ω, $L(x) \le p(x) \le U(x)$ for every string. A pair with $x \succ y$ has $p(x) - p(y) \ge \hat{p}(x) - \hat{p}(y) - E_{xy} > 0$. For the threshold query, $L(x) > \tau$ implies $p(x) > \tau$, and $U(x) < \tau$ implies $p(x) < \tau$.

For the top-k query, every $y \in \mathrm{Below}(x)$ has $p(y) < p(x)$, either because $p(y) \le U(y) < L(x) \le p(x)$ or because y is shown below x. Hence $|\{y \neq x : p(y) \ge p(x)\}| \le 2^m - 1 - |\mathrm{Below}(x)| \le k-1$ for an IN string, which is therefore a sure member. Symmetrically, every $y \in \mathrm{Above}(x)$ has $p(y) > p(x)$, so an OUT string has at least k strictly larger strings and is a sure non-member. A sure member is never a sure non-member, so the two tests never hold together, and no refined interval is empty. The argument is deterministic given Ω and uses no property of the strings that an algorithm evaluates, so it covers all 2^m strings. □

B. Search without Materialization

The rules refer to all 2^m strings, but evaluating every string one by one is infeasible for wide outputs. The search below therefore finds the strings whose labels need their own bounds, and it labels all other strings OUT together.

Branches. We order the m output bits block by block, and within a block from the highest bit to the lowest. A *branch* B fixes a prefix of d bits in this order and stands for the 2^{m-d} strings that complete it. For every group g, the blocks fixed completely by B contribute their factor intervals at the fixed values. The block whose high bits are fixed contributes the hull $[\min \underline{u}_{i,g}(y_i), \max \overline{u}_{i,g}(y_i)]$ over the values y_i that agree with those bits, and every free block contributes the hull over all its values. The branch bound $\overline{U}(B)$ evaluates the upper end of (3) with these intervals and the arithmetic of Lemma 1. The hulls of all bit prefixes of a factor table with 2^{m_i} entries form a pyramid with fewer than 2^{m_i+1} entries, which pairwise minima and maxima build in one pass per level. For example, for a block with $m_i = 2$ whose high bit is fixed to 1, the hull covers the block values 10 and 11. This rule is the interval counterpart of the branch bound in QCut's exact `top()` search [4], which bounds a branch of the point estimate by the largest magnitude that each unassigned block can contribute.

Lemma 2 (Branch bound)**.** *For every branch B, $\overline{U}(B) \ge \max_{x \in B} U^0(x)$.*

Proof sketch. For every completion x of B, each factor interval of x lies inside the interval that B uses for the same factor. Interval operations preserve containment, so the interval of $p(x)$ lies inside the interval computed for B. □

The implementation widens $\overline{U}(B)$ by a relative slack of 10^{-12}, which covers the different floating-point rounding of branch and string evaluation.

Collection. Algorithm 1 walks the branches level by level and keeps the children whose branch bound reaches a threshold θ. At the last level, it computes L^0 and U^0 of the surviving strings with the same arithmetic as enumeration, and it returns the strings with $U^0(x) \ge \theta$. The pilot selects the set A with the same machinery. On the pilot estimate every interval has width zero, so the search finds the strings with the largest pilot scores.

Top-k query. Let $L^0_{(k)}$ be the k-th largest value of L^0 over all 2^m strings. The search answers the query in four steps. (i) First, a beam pass walks the branches level by level but keeps only the 4096 branches with the largest $\overline{U}$ at every level. Among the strings it reaches, the k-th largest L^0 gives a lower bound $\lambda \le L^0_{(k)}$, and $\lambda = -\infty$ if it reaches fewer than k strings. (ii) Then Algorithm 1 runs with $\theta = \min\{\lambda, \min_{x \in A} L(x), \min_{x \in A} U(x)\}$ and returns the set $\mathcal{C}$. Every string with $L^0(x) \ge L^0_{(k)}$ has $U^0(x) \ge L^0(x) \ge \lambda \ge \theta$, so it is collected into $\mathcal{C}$. As a result, $L^0_{(k)}$ can be read exactly from $\mathcal{C}$. (iii) Next, let $\theta_k = \min\{L^0_{(k)}, \min_{x \in A} L(x), \min_{x \in A} U(x)\}$ and $\mathcal{C}_k = \{x \in \mathcal{C} : U^0(x) \ge \theta_k\}$. Because every $x \in A$ has $U^0(x) \ge U(x) \ge \theta_k$, we have $A \subseteq \mathcal{C}_k$. (iv) Finally, inside $\mathcal{C}_k$, the search applies the rules of Section IV-A with the refined bounds and the pair tests. It counts only strings of $\mathcal{C}_k$, and it labels a string IN only if also $L(x) \ge \theta_k$. Every string outside $\mathcal{C}_k$ has the k strings of largest L^0 as witnesses and is labelled OUT. A batch of queries with several values of k and τ can share one collection, with θ set to the minimum of the thresholds that the queries need, because the candidate set of each query is obtained by filtering $\mathcal{C}$.

Threshold query. The search collects $\mathcal{C}_\tau = \{x : U^0(x) \ge \min\{\tau, \min_{x' \in A} U(x')\}\}$, applies the threshold rule to its strings, and labels every other string OUT.

Algorithm 1 Collection of the strings with $U^0(x) \ge \theta$

Require: factor intervals, threshold θ
$\mathcal{F} \leftarrow \{$the empty prefix$\}$
for $d = 1, \ldots, m$ **do**
  $\mathcal{F} \leftarrow$ the two children of every branch in $\mathcal{F}$
  if $d < m$ **then**
    $\mathcal{F} \leftarrow \{B \in \mathcal{F} : \overline{U}(B) \ge \theta\}$
  end if
end for
compute $L^0(x), U^0(x)$ for every $x \in \mathcal{F}$
return $\{x \in \mathcal{F} : U^0(x) \ge \theta\}$ with their bounds

Theorem 4 (Search equals enumeration)**.** *For every top-k query and every threshold query, the search returns, for every one of the 2^m strings, the label that the rules of Section IV-A assign when they are applied to all 2^m strings with the same bounds L, U and the same pair tests. The search terminates after m levels, it computes branch bounds only for children of the branches it keeps, each at a cost of $O(|\mathcal{G}|\, s)$ interval operations, and it stores one level of branches at a time.*

Proof sketch. Algorithm 1 discards a branch only if $\overline{U}(B) < \theta$, so by Lemma 2 it returns exactly $\{x : U^0(x) \ge \theta\}$. We compare the top-$k$ labels with enumeration in three cases. Outside $\mathcal{C}_k$, a string x is not in A, so $U(x) = U^0(x) < \theta_k \le L^0_{(k)}$. The k strings of largest L^0 have $L \ge L^0 \ge L^0_{(k)} > U(x)$ and differ from x, so $|\mathrm{Above}(x)| \ge k$ and enumeration labels x OUT. Inside $\mathcal{C}_k$, every string has $U(x) \ge \theta_k$, while every string y outside has $L(y) \le U(y) < \theta_k$. So no outside string enters $\mathrm{Above}(x)$, and the pair tests involve only strings of $A \subseteq \mathcal{C}_k$. If also $L(x) \ge \theta_k$, every outside string lies in $\mathrm{Below}(x)$, and

the counts inside $\mathcal{C}_k$ equal those of enumeration. If instead $L(x) < \theta_k$, then $x \notin A$ and x has no pair test. The k strings of largest L^0 then have $U \geq L^0_{(k)} \geq \theta_k > L(x)$, so at least k competitors remain, and enumeration does not label x IN either. For the threshold query, a string outside $\mathcal{C}_\tau$ is not in A and has $U(x) = U^0(x) < \tau$, so both procedures label it OUT. Inside, the rule depends on the string alone. The empty-interval case also depends only on A, so it is the same in both procedures. Finally, the walk has m levels, and every level holds finitely many branches, so the search terminates. □

Theorems 3 and 4 together solve the problem of Definition 1. The search returns the strings of $\mathcal{C}_k$ (or $\mathcal{C}_\tau$) with their labels, and it labels all other strings OUT. With probability at least $1 - \delta$, every IN and OUT label in this answer is correct, including the labels of the strings that the search never evaluates. For example, for the top-10 query of Fig. 1, the search collects 1,354 candidate strings out of 2^{24}.

V. Appended Shots

Assumption 1(A3) requires the shot counts to be fixed before the inference shots are seen. In practice, however, a user may look at the answer, see UND strings near the boundary, and ask for more shots, preferably on the subexperiments that decide the boundary. If the old bounds were simply rechecked on the accumulated data after more shots, the level δ would break, because the counts at which a fixed-sample bound is evaluated would then depend on the data. This section gives a protocol that keeps δ under any data-dependent allocation and stopping rule (Theorem 5). It shows that every string with a positive margin is eventually decided and how many shots suffice (Theorem 6), and it states the allocation rule that directs appended shots to the boundary.

A. Rounds with Fresh Batches

Round 0 runs a pilot D_0. Then, in round $t = 1, 2, \ldots$, the procedure

1) chooses positive batch sizes $n_{e,t}$, as any function of the data $D_0, \ldots, D_{t-1}$ observed so far.
2) runs subexperiment e another $n_{e,t}$ times, which gives a fresh batch D_t.
3) computes the bounds of Section III from D_t alone at level $\delta_t = \delta\, 2^{-t}$, split into the shares of Section III in fixed proportions, with the earlier data $D_0, \ldots, D_{t-1}$ in the role of the pilot (it selects the strings A and the pairs P).
4) labels the strings by the rules of Section IV-A from the running bounds $L^*_t(x) = \max_{r \leq t} L_r(x)$ and $U^*_t(x) = \min_{r \leq t} U_r(x)$ and from all pairs shown so far.
5) stops or continues, by any rule.

The batch sizes double from round to round, that is, $\sum_e n_{e,t} = 2^{t-1} \sum_e n_{e,1}$, so the last batch holds at least half of all inference shots. A procedure that commits in advance to one inference round may spend the whole level δ in it, and it is then the fixed-sample procedure of Sections III and IV with a pilot.

Theorem 5 (Appended shots)**.** *Under Assumption 1*(A1, A2)*, for a top-k or threshold query and for any rule that chooses the batch sizes and the stopping round from the data observed before each round, every* IN *and* OUT *label issued in any round is correct with probability at least $1 - \delta$. Moreover, with probability at least $1 - \delta$, every* IN *or* OUT *label issued in some round is also issued in every later round.*

Proof sketch. Let Ω_t be the event of Theorem 2 for round t. Given the data before round t, the batch sizes, the selected strings and the pairs of round t are fixed, and D_t consists of new independent shots. Theorem 2 therefore holds conditionally, which gives $\Pr[\Omega_t^c] \leq \delta_t$. A union bound over the rounds gives $\Pr[\bigcap_t \Omega_t] \geq 1 - \sum_t \delta_t \geq 1 - \delta$. Rounds after the stopping round can be defined by any fixed continuation, so the bound covers every stopping rule. On $\bigcap_t \Omega_t$, every $L_r(x) \leq p(x) \leq U_r(x)$, so the running bounds also enclose $p(x)$, and every pair shown in any round is correct. Theorem 3 uses only these two facts, so every label is correct. Moreover, as t grows, L^*_t rises, U^*_t falls and the set of pairs grows, so $|\mathrm{Below}(x)|$ and $|\mathrm{Above}(x)|$ can only grow. Hence, on $\bigcap_t \Omega_t$, an IN or OUT label is never withdrawn. □

The key to this argument is that each round evaluates its bounds once, on its own batch, at batch sizes fixed before the batch is drawn. This keeps the argument to one conditioning step, and it allows every round to use the refined bounds of Section III-C, even though their score and pair constraints combine all subexperiments.

B. Eventual Decision and Shot Budget

Every factor in (3) is a linear functional of one distribution with coefficients in $\{-1, 0, 1\}$, so its value lies in $[-1, 1]$ on the region. For factors with values in $[-1, 1]$, the range of a product is at most the sum of the factor widths. With $W_e = \sum_z (\overline{q}_{e,z} - \underline{q}_{e,z})$ the total width of the region of subexperiment e, Lemma 1 therefore gives, for every string,

$$U(x) - L(x) \leq \Delta := \kappa\, \bar{s} \max_e W_e, \quad \kappa = (1 + 2^{-m}) \textstyle\sum_g |c_g|, \tag{10}$$

where $\bar{s}$ is the largest number of factors in a group. For N shots, a Clopper-Pearson interval at level α lies within $\sqrt{\ln(2/\alpha)/(2N)}$ of the empirical frequency, so $W_e \leq |Z_e| \sqrt{2 \ln(2|\mathcal{S}||Z_e|/\delta_0)/N_e}$.

Theorem 6 (Eventual decision)**.** *On Ω, every string with $|p(x) - \tau| > \Delta$ is decided by the threshold query. If $k < 2^m$ and the gap $\gamma = p_{(k)} - p_{(k+1)}$ between the k-th and $(k+1)$-th largest values of p is positive and $\Delta < \gamma/2$, every string is decided by the top-k query. With batch sizes $n_{e,t} \geq \beta\, 2^{t-1} n_1$, where $n_1 = \min_e n_{e,1}$ and $\beta > 0$ is a constant, and the fixed shares of step 3, Δ tends to 0 over the rounds, so on $\bigcap_t \Omega_t$ every string with $p(x) \neq \tau$, and for $\gamma > 0$ every string of the top-k query, is decided after finitely many rounds.*

Proof sketch. On Ω, $L(x) \geq p(x) - \Delta$ and $U(x) \leq p(x) + \Delta$. These bounds decide x for the threshold query when $|p(x) - \tau| > \Delta$. For the top-k query with $\Delta < \gamma/2$, the lower bound

of each of the k largest strings exceeds the upper bound of every other string, which makes the former IN and the latter OUT. In round t, N_e grows like 2^t while $\ln(1/\delta_t)$ grows like t, so Δ tends to 0. □

Inverting (10) gives a sufficient shot budget for an UND string whose true gap to the boundary is known to be at least a given value, and the extended version [11] states it. The refined bounds of Section III-C can only decide strings earlier, and the experiments measure how much earlier.

C. Allocation to the Boundary

Theorem 5 allows any allocation that depends only on the data of earlier rounds. Our rule spends the shots of round t on the subexperiments that shrink the uncertainty of the comparisons that are still open. Specifically, from the pooled data of the earlier rounds, it ranks the strings by $\hat{p}$, takes the eight strings at ranks $k-5, \ldots, k+2$, and for each pair (x, y) of them computes the first-order influence of one shot of subexperiment e on $\hat{p}(x) - \hat{p}(y)$. The weight a_e of subexperiment e is the sum over the pairs of the variance of this influence. Every subexperiment first receives half of its uniform share, so that Theorem 6 applies, and the other half of the batch is split in proportion to $\sqrt{a_e}$ (Neyman allocation), with largest-remainder rounding. One allocation serves all queries of a round. A variance-minimizing split in the style of ShotQC [14] weights each subexperiment by the variance of the estimate over all 2^m strings. In contrast, our weights depend on the query and ignore strings far from the boundary. With the pilot as round 0 and a single further round, this rule is the allocation of our method in Section VI, where it is compared with the uniform split.

VI. Experimental Evaluation

The evaluation answers four questions. How often are the IN and OUT labels wrong in our runs, compared with the level of Theorem 3 (Section VI-B)? How many strings do our labels decide at a given number of shots, compared with sound alternatives (Section VI-C)? What does each component contribute (Section VI-D)? Does the search scale beyond enumeration (Section VI-E)?

A. Setup

Circuits. The benchmark was specified and its success rule fixed before any instance was built. It has three families, and no generated instance was excluded. *Arithmetic*: a ripple-carry adder [15] on 16, 20 and 24 qubits, all measured, with four input bits prepared in superposition by random rotations, so that 16 outputs carry graded probabilities. Two wire cuts on one carry wire split it into two subcircuits. *QAOA*: depth-2 QAOA for MaxCut on two random 3-regular clusters joined by one edge, with 20 and 24 qubits, angles optimized on the uncut circuit, and the crossing gate cut in both layers (two gate cuts). *Bernstein-Vazirani*: 16 and 20 data qubits with one wire cut on the ancilla. It has a single dominant string and serves as a sanity check. Each arithmetic and QAOA size has ten instances from seeds. Five are development instances, on which methods were tuned, and five are held-out instances, whose reference distributions stayed sealed until the method and its parameters were frozen. Cutting, the subexperiments and the estimator are those of QCut [4] with exact expansion. The reference distribution of each instance is the reconstruction at the exact subexperiment distributions. It equals the uncut state-vector distribution to within 10^{-10} for every instance. We also checked the exact subexperiment distributions of 13 instances against 10^5 or 10^6 shots per subexperiment sampled with the circuit simulator Qiskit Aer. They include every size, with a held-out seed for every size except QAOA with 24 qubits, and all 13 pass χ^2 tests with a Bonferroni-adjusted minimum p-value of at least 0.01.

Methods. All methods use the labelling rules of Section IV and differ in the bounds that they feed into these rules. At $m \leq 24$, we compute the labels of BL1 and R-CP by enumeration and those of our method by the search, with enumeration as a fallback when the collected set grows large. By Theorem 4, search and enumeration give the same labels. *Ours* combines the region of Section III-A with the refinement and the tighter terms of Section III-C. Its pilot uses 5% of the shots, and the remaining shots go to the boundary in a single round (Section V-C). Its parameters are $|A| = 128$ and the level split of Section III-C. The pairs P of a top-k query are all pairs among the pilot ranks $\max(1, k-5)$ to $k+2$. One allocation, computed from the pairs of the top-10 query, serves all four queries of a run, and each run spends the whole level δ of every query in its single inference round. The extended version [11] specifies the pilot selection, the allocation weights and the radii of BL1 in full. The variants in Section VI-D use the refinement without the tighter terms ($|A| = 32$, $\delta_0 = 0.005$, $\delta_s = 0.005$, $\delta_p = 0.04$), a uniform split of the shots after the pilot, or both changes. *BL1* is a uniform bound in the style of Neumann et al. [5]. It bounds the reconstruction error of every string by Hoeffding's inequality or the Dvoretzky-Kiefer-Wolfowitz inequality [16] with a union bound, and it reports the stronger of the two for each configuration. *R-CP* combines the per-category Clopper-Pearson region of Section III-A with interval propagation and no refinement, and it is the strongest simple sound rule we know. QCut's `top()` on the estimate serves as the uncertified reference for answers and cost. Before any held-out result was computed, we froze the method and its parameters on the development instances, choosing the variant that met the success rule on the most development instances (Section VI-D).

Protocol. Every comparison of the benchmark is at an equal total number of shots $B = |\mathcal{S}|N$, where N is the number of shots per subexperiment of a uniform split. Our method spends its pilot and its allocated shots inside B. We use $\delta = 0.05$, the top-5 and top-10 queries and thresholds $\tau \in \{0.01, 0.05\}$, and independent seeds per repetition. On the development instances, $N \in \{10^3, 10^4, 10^5, 10^6\}$ with 10 to 30 repetitions. On the held-out instances, $N = 10^5$ ($B \approx 5 \cdot 10^6$, the largest grid budget within $B \leq 10^7$) with 10 repetitions. A string is a *sure member* of the top-k answer if fewer than k other strings have a probability at least as large, up to a relative tolerance

TABLE II
RUNS WITH A WRONG IN OR OUT LABEL IN ANY OF THEIR FOUR QUERIES (ERR.) AND THE 95% UPPER LIMIT OF THE ERROR RATE (UP.).

	Development			Held-out		
Bounds	Runs	Err.	Up.	Runs	Err.	Up.
BL1	1,988	0	0.0015	266	0	0.0112
R-CP	1,988	0	0.0015	267	0	0.0112
Refined, uniform	1,987	0	0.0015			
Refined, boundary	1,987	0	0.0015			
Tighter, uniform	1,042	0	0.0029			
Tighter, boundary (ours)	1,042	0	0.0029	264	0	0.0113

of 10^{-12} for ties, and a *sure non-member* if at least k strings have a larger probability beyond that tolerance. Strings tied at the boundary are neither. We report the share of sure members labelled IN, the share of sure non-members labelled OUT, and wrong labels. An instance *succeeds* if its top-10 answers meet three conditions at some evaluated budget $B \leq 10^7$. First, the mean IN share is at least 0.5. Second, the mean OUT share is at least 0.999. Third, the IN share is at least twice BL1's at the same B or at least BL1's at a budget at least four times larger. If BL1 labels no member IN at any evaluated budget, any positive share meets the third condition. A family succeeds if at least 60% of its held-out instances succeed.

Comparison fairness. In every configuration, all methods receive the same total number of shots and use the same decision rules and, where it applies, the same search. Our method includes its pilot and its allocation to the boundary within this budget, and Table IV separates the effects of the bounds and of the allocation. BL1 takes the better of its two inequalities for every configuration, and R-CP spends the whole level δ on its region. Our method was tuned on the development instances only.

Platform and artifacts. The code, the result rows of all runs and the extended version are public [11]. All runs use one thread of an Intel Core i9-13900K with 64 GB of memory, Python 3.11, NumPy 2.4 and SciPy 1.17, on a machine shared with other jobs.

B. Correctness of the Labels

No IN or OUT label of any sound method is wrong in our runs. Table II counts the runs in which at least one of the $4 \cdot 2^m$ labels of the four queries is wrong. Over the 10,831 runs of all sound methods on development and held-out instances, 0 runs contain a wrong label, and for our method the count is 0 of 1,306. A run counts as an error if any of its four queries has a wrong label, so this criterion is stricter than the guarantee of level $\delta = 0.05$ per query. On the development instances, where each method has more than 10^3 runs, the upper confidence limits of the error rate lie below 0.003, far below δ, and the held-out runs, which include the runs at every evaluated budget, add no error. These results show that the bounds are conservative on the evaluated circuits. The next sections measure what this conservatism costs in decided strings.

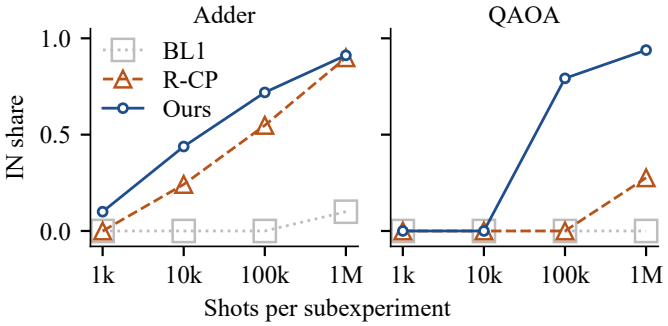


Fig. 2. Development instances: share of sure top-10 members labelled IN against shots per subexperiment.

C. Decided Strings at Equal Shots

On the held-out QAOA circuits, our labels decide most sure top-10 members, where R-CP decides almost none and BL1 none. At $B \approx 5 \cdot 10^6$ total shots, our method labels 0.74 of the sure top-10 members IN on QAOA and 0.67 on arithmetic, with mean OUT shares of 1.00000 and 0.99995. R-CP labels 0.02 and 0.58, and BL1 decides no string of this query at this budget (Table III). By the success rule, our method succeeds on 9 of the 10 held-out QAOA instances and on 15 of the 15 held-out arithmetic instances, so it succeeds on both families. R-CP succeeds on 0 and 15 of them. On arithmetic, R-CP meets the success rule as well, because BL1 labels no member IN, and the comparison clause of the rule then only asks for a positive share. Even so, our labels decide a larger share there. On QAOA, the success rule separates the two methods.

BL1 bounds every string by one deviation bound for the whole estimate. With 2^m strings and the large coefficients of the reconstruction, this deviation is of the order of the probabilities of the top strings at $N \leq 10^5$, so its intervals do not separate the top-10 boundary. For the top-10 query, BL1 therefore issues neither IN nor OUT labels at these budgets. Only at $N = 10^6$ does it start to label a few arithmetic top-10 members IN (Fig. 2), and for the threshold queries it labels an occasional string IN at $N = 10^5$. In contrast, R-CP labels almost all sure non-members OUT, because their upper bounds fall below the probabilities of the top strings. On QAOA, however, the intervals of R-CP for the top strings overlap. The top strings come in groups of tied and near-tied strings, because the state is invariant under flipping all bits, and flipping one cluster changes only the term of the crossing edge. The interval propagation of Lemma 1 then widens the interval of every product of factors until the intervals of different groups overlap as well. The refinement bounds the scores directly, and its pair tests separate strings whose individual intervals overlap.

Across budgets on the development instances (Fig. 2), our labels decide at least as many members as R-CP and BL1, and the gap is largest on QAOA. On QAOA, our method starts to label members IN at $N = 10^5$, while R-CP needs $N = 10^6$. At $N = 10^6$ shots per subexperiment, our labels decide 0.94 of the sure QAOA top-10 members, against 0.28 for R-CP

TABLE III
HELD-OUT INSTANCES, TOP-10 QUERY, $N = 10^5$: MEAN IN SHARE ($\pm$ STANDARD DEVIATION OVER RUNS), MEAN OUT SHARE, AND INSTANCES MEETING THE SUCCESS RULE.

		BL1	R-CP			Ours		
Circuit	Inst.	IN	IN	OUT	Succ.	IN	OUT	Succ.
Adder, $m = 16$	5	0.00	0.62 ± 0.07	0.99972	5/5	0.66 ± 0.08	0.99986	5/5
Adder, $m = 20$	5	0.00	0.56 ± 0.05	0.99998	5/5	0.69 ± 0.08	0.99999	5/5
Adder, $m = 24$	5	0.00	0.57 ± 0.05	1.00000	5/5	0.65 ± 0.08	1.00000	5/5
QAOA, $m = 20$	5	0.00	0.00 ± 0.00	0.99987	0/5	0.72 ± 0.21	1.00000	4/5
QAOA, $m = 24$	5	0.00	0.04 ± 0.08	0.99998	0/5	0.76 ± 0.15	1.00000	5/5

TABLE IV
ABLATION ON DEVELOPMENT INSTANCES (12 ARITHMETIC, 10 QAOA), $N = 10^5$: MEAN TOP-10 IN AND OUT SHARES.

	Arithmetic		QAOA	
Bounds	IN	OUT	IN	OUT
BL1	0.00	0.00000	0.00	0.00000
R-CP	0.55	0.99991	0.00	0.99993
Refined, uniform	0.73	0.99996	0.21	0.99998
Refined, boundary	0.69	0.99995	0.47	0.99999
Tighter, uniform	0.76	0.99996	0.52	1.00000
Tighter, boundary (ours)	0.72	0.99996	0.79	1.00000

TABLE V
SEARCH ON QAOA, TOP-10: STRINGS EVALUATED, IN AND UND COUNTS, TIMES IN SECONDS, AND EXTRAPOLATED ENUMERATION TIME IN DAYS.

m	N	Evaluated	Share	IN	UND	Ours	`top()`	Enum.
32	10^5	20,164	4.7e-6	0	839	2.8	0.02	0.1
32	10^6	17,910	4.2e-6	2	25	2.8	0.03	0.1
36	10^5	19,538	2.8e-7	0	711	6.6	0.09	1.4
36	10^6	17,470	2.5e-7	6	6	6.5	0.25	1.3
40	10^5	8,599,804	7.8e-6	0	3,328,024	455.5	0.42	20.3
40	10^6	18,404	1.7e-8	0	112	21.6	1.06	19.5
40	10^7	17,718	1.6e-8	7	11	22.9	1.61	–

and 0.00 for BL1. On arithmetic, R-CP catches up with our method at this budget (0.90 against 0.91).

D. Ablation

On the development QAOA instances at $N = 10^5$, the share of sure members labelled IN rises from 0.00 with R-CP to 0.21 with the refinement, then to 0.52 with the tighter terms, and finally to 0.79 when the tighter terms are combined with allocation to the boundary (Table IV). The row of the tighter terms introduces three changes at once, namely the terms (i) to (iii), the larger selected set ($|A| = 32 \rightarrow 128$) and the level split, so this row measures their combined effect. On arithmetic, the refinement contributes most ($0.55 \rightarrow 0.73$). The tighter terms add a little, and allocation to the boundary lowers the share slightly ($0.76 \rightarrow 0.72$), so the uniform split decides slightly more members on this family. We froze the variant with boundary allocation because it met the success rule on all 10 development QAOA instances, against 7 for the uniform split, and both variants met the rule on all 12 development arithmetic instances of Table IV, which are those with ten repetitions of all six variants at $N = 10^5$.

The changes target different parts of the radius (8). In a diagnosis on three development QAOA instances with 20 qubits ($N = 10^5$, one repetition each), about 40% of the radius of a selected score came from the remainder R_f, which terms (i) and (iii) target. In the same runs, competitors outside the 32 selected strings kept propagated upper bounds above the top scores, which the larger selected set replaces by refined bounds.

E. Search beyond Enumeration

The search answers queries over spaces far beyond the reach of enumeration. At $m = 40$ output qubits, a space of $2^{40} \approx 1.1 \cdot 10^{12}$ strings, it evaluates fewer than $2 \cdot 10^4$ strings exactly at $N \geq 10^6$ and returns the labels in about 22 seconds, while full enumeration would take about 20 days on the same machine (Table V). QCut's `top()` on the same estimate takes 1 to 2 seconds but certifies nothing. No label of the search is wrong against the exact answer, which the bit-level search computes from the exact factors without the 2^m vector. At the lower budget of $N = 10^5$ and $m = 40$, the bounds are too wide to decide the boundary, so more strings remain UND, and the search evaluates $8.6 \cdot 10^6$ strings, about a $7.8 \cdot 10^{-6}$ share of the space. With 10^7 shots per subexperiment, 7 of the top-10 strings are labelled IN and 11 strings remain UND among 2^{40}.

The search runs use QAOA instances of the same construction with $m \in \{32, 36, 40\}$, one seed for each size, and these instances are not part of the benchmark above. Every subexperiment receives a pilot of $N/20$ shots in addition to its N uniform inference shots, and the top-10 query uses our bounds with the tighter terms. The time of our method includes the pilot selection and the certification, and the enumeration time is extrapolated from a measured chunk. Each configuration ran once.

VII. RELATED WORK

Circuit cutting. Wire cuts and gate cuts write a large circuit as a quasi-probability combination of the results of subcircuit runs [1], [2]. CutQC [3] evaluates large circuits on small devices and locates the probable outputs by dynamic definition, which recursively zooms into the more probable bins of the output. TensorQC [17], in turn, screens subcircuit states by their L2 norm. Both methods search for probable strings efficiently, but neither was designed to attach a confidence level to its answer. QCut [4] uses branch and bound over the product form to answer top-k queries exactly for

the estimate, without building the 2^m entries. Our search keeps this skeleton and replaces the point values by interval bounds (Section IV-B), and QCut's `top()` serves as the cost reference in Section VI. The error analyses of cutting that are closest to our problem address values and variances. Neumann et al. [5] bound the sampling error of the reconstructed probabilities with Hoeffding's inequality and a union bound. ShotQC [14] propagates the variances and covariances of subcircuit estimates and allocates shots to reduce the variance of the output. Daskin [18] derives error bounds for a correlation-based cutting procedure from quantum-classical moment duality. Our work builds on value bounds of this kind and adds a query layer with three-valued labels on top of them, so the evaluation gives a uniform Hoeffding-type bound our rules and search. Quantum computers have also been applied to database problems, for example to multiple query optimization [19]. We study the opposite direction, query processing over the outputs of quantum circuits. Distributed quantum computing runs one computation on several devices that exchange quantum communication, and our prior work reduces this communication [20]. Cutting instead replaces the communication by classical reconstruction.

Top-k queries over uncertain data. Top-k processing prunes candidates with score bounds, as in the threshold algorithm [21] and the techniques surveyed by Ilyas et al. [22]. For uncertain data, U-Topk [6] and PT-k [8] define top-k answers under possible-worlds semantics, where the probabilities of the tuples are part of the input. Ré et al. [7] estimate the probabilities of query answers by Monte Carlo simulation and spend simulation steps on the answers whose intervals still overlap the top-k boundary. They sample a known probabilistic database, and each answer probability is estimated by its own simulation. Dylla et al. [9] answer top-k queries over non-materialized views of a probabilistic database. They keep lower and upper bounds on the probabilities of the answers, hold the current top-k answers by their lower bounds, and keep an answer as a candidate only if its upper bound exceeds the worst lower bound among the current top-k answers. Our search uses the same bound-driven pruning, but the probabilities mean something different. In a probabilistic database, the probabilistic model is given, and the query computes or estimates answer probabilities from it. In our setting, the scores are the values of an unknown distribution that is reconstructed from samples through signed factors, and the level $1 - \delta$ is coverage over the random shots. An UND label therefore reflects limited data, and appended shots narrow the bounds that produced it.

Approximate query processing. Online aggregation [23] reports running estimates of an aggregate with confidence intervals while it samples, and approximate query processing systems attach error bars to sampled answers and check their reliability [24]. These systems bound aggregates over sampled tuples. In contrast, our estimates are signed products of shared sampled tables, and our answers are rankings of 2^m values that are never computed one by one.

Confidence-bound identification. LUCB [25] selects a set of nearly best arms of a stochastic bandit with confidence bounds and adaptive sampling, and it stops when the largest excluded upper bound exceeds the smallest selected lower bound by less than a tolerance. Réda et al. [26] study best-arm identification when the arm means are linear in shared parameters. In data engineering, adaptive sampling also selects a group of K nodes with approximately maximum group betweenness centrality [27]. Our decision rules follow the same separation principle. However, the arms of our problem are 2^m bitstrings that are never sampled one at a time, and their scores are signed multilinear functions of shared subexperiment distributions. One shot therefore moves the bounds of many strings, and most strings receive their labels through branch bounds without being evaluated. Time-uniform confidence sequences [28] keep a guarantee under data-dependent sampling and stopping. Our rounds obtain this property with fresh batches and a geometric split of δ, which lets every round use the refined bounds of Section III-C (Theorem 5). With deterministic bounds, Sui and Tang [29] return the exact top-k set from few evaluations once the lower and upper bounds leave only one possible set. In contrast, our bounds are statistical and hold with probability at least $1 - \delta$.

Factorized query processing. The reconstruction is a sum-product query over small factor relations in the sense of FAQ [30], a framework whose authors write that it deals "solely" with exact computation [31, p. 9]. Factorized databases [32] store and query relations as products and unions of small relations without materializing them. Ranked enumeration [33] returns the answers of a full conjunctive query one by one in ranking order. All these lines evaluate queries over given input relations. In our representation, the factor relations are signed, and their entries are estimated from shots. The ranking order itself is therefore uncertain, and an answer has to state which part of the order holds with probability at least $1 - \delta$. Our search can thus be read as ranked access to a signed sum-product query whose factors are intervals.

VIII. Conclusion

Circuit cutting represents the distribution over all output strings as a sum of products of tables with signs, and the tables are estimated from shots. Current tools rank the estimate and do not indicate which of the returned strings may be wrong. We answer top-k and threshold queries directly on this representation, and under the stated sampling model, with a probability chosen by the user, the in and out labels of all strings are correct at the same time. One confidence region over the raw measurement categories bounds every string, and Bernstein bounds tighten the bounds at the boundary of the answer. A branch-and-bound search returns the same labels as full enumeration without materializing the distribution, and when shots are appended in fresh batches, the guarantee still holds under any data-dependent allocation. On held-out QAOA circuits, our method labels on average 74% of the sure top-10 members IN, against 2% for per-category Clopper-Pearson bounds, at a budget where a uniform bound decides no string

of this query. At 40 output qubits, the search evaluates at most a $7.9 \cdot 10^{-6}$ share of the strings at the tested budgets.

APPENDIX A
FULL PROOFS

This appendix proves every theorem and lemma of the paper in full. We first state the two probability tools that the proofs use. All probabilities are conditional on the pilot when one is used. Under Assumption 1, the pilot fixes the selected strings A, the pairs P, the error shares and the shot counts N_e, and the inference shots are fresh. A bound that holds conditionally on every pilot outcome also holds after averaging over the pilot, so every statement below also holds unconditionally.

A. Shot Budget

Inverting (10) gives a sufficient shot budget for an UND string. Let $|Z| = \max_e |Z_e|$. If the true gap of a string satisfies $|p(x) - \tau| \geq g > 0$, then on Ω more than

$$N(g) = 2\ln(2|\mathcal{S}||Z|/\delta_0)\,(\kappa\bar{s}|Z|/g)^2$$

shots per subexperiment give $\Delta < g$ and decide the string, by the proof of Theorem 6. For the top-k query with boundary gap $\gamma > 0$, the budget is $N(\gamma/2)$. In practice the observed gap $\hat{g} = |\hat{p}(x) - \tau|$ serves as a guess for g, so the budget is conditional on the gap. On Ω, a string with $p(x) = \tau$, or one in a tie that straddles positions k and $k+1$, stays UND for every number of shots.

B. Experimental Configuration

Pilot and selected strings. Each subexperiment receives $\lfloor N/20 \rfloor$ pilot shots from a separate random stream. The pilot estimate defines zero-width factor intervals, and the top-k search of Section IV-B with $k = |A|$ returns the $|A|$ strings with the largest pilot scores, ties broken by string index ($|A| = 128$ for our method and 32 for the variants without the tighter terms).

Pairs. For a top-k query, P contains all pairs among the strings at pilot ranks $\max(1, k-5), \ldots, k+2$, so $|P| = 28$ for $k = 10$ and $|P| = 21$ for $k = 5$. These strings belong to A.

Allocation. For each pair (x, y) of the top-10 query, let $w_{e,xy}(z)$ be the weight (7) of the difference $p(x) - p(y)$ evaluated at the pilot estimate. The weight of subexperiment e is $a_e = \sum_{(x,y)} \mathrm{Var}_{\hat{q}_e^{\mathrm{pilot}}}[w_{e,xy}(Z)]$. Of the $B - |\mathcal{S}|\lfloor N/20 \rfloor$ inference shots, every subexperiment first receives $\lceil u_e/2 \rceil$, where u_e is its share of a balanced uniform split, and the rest is split in proportion to $\sqrt{a_e}$ with largest-remainder rounding, ties broken by index. The same counts serve all four queries of a run. Each query spends its whole level $\delta = 0.05$ in this single inference round, split as stated in Section III-C.

BL1. BL1 gives every string the same half-width D around $\hat{p}(x)$. If every factor value and mass deviates from its true value by at most ε, a product of r factors with values in $[-1, 1]$ deviates by at most $r\varepsilon$, so $D = \sum_g |c_g| r_g \varepsilon + 2^{-m} \sum_g |c_g| r_g \varepsilon_S$, where r_g is the number of factors of group g and ε_S is the deviation of a mass. Let $N_{\min} = \min_e N_e$. The Hoeffding variant uses $\varepsilon = \varepsilon_S = \sqrt{2\ln(2|\mathcal{F}_{\mathrm{all}}|/\delta)/N_{\min}}$ with a union bound over all $|\mathcal{F}_{\mathrm{all}}|$ distinct factor functionals. The DKW variant spends $\delta/2$ on the Massart form of the Dvoretzky-Kiefer-Wolfowitz inequality [16], split over the $|\mathcal{S}|$ subexperiments, which bounds every category frequency by $2\varepsilon_D$ with $\varepsilon_D = \sqrt{\ln(4|\mathcal{S}|/\delta)/(2N_{\min})}$. A factor value of subexperiment e with 2^{r_e} supported categories then deviates by at most $\min(2, 2^{r_e} \cdot 2\varepsilon_D)$, and the masses use Hoeffding's inequality at level $\delta/2$. BL1 reports the variant that decides more strings in each configuration.

C. Two Tools

Lemma A.1 (Bernstein's inequality)**.** Let $X_1, \ldots, X_n$ be independent real random variables with $\mathbb{E}X_j = 0$ and $|X_j| \leq b$ almost surely, and let $V \geq \sum_j \mathbb{E}X_j^2$. Then, for every $t > 0$,

$$\Pr\Big[\Big|\sum_j X_j\Big| > \sqrt{2Vt} + \frac{bt}{3}\Big] \leq 2e^{-t}.$$

Proof. If $V = 0$, every X_j vanishes almost surely and the claim holds. Otherwise, let $0 \leq \lambda < 3/b$. Because $\mathbb{E}X_j = 0$, $|X_j| \leq b$ and $k! \geq 2 \cdot 3^{k-2}$ for $k \geq 2$, the power series of the exponential gives

$$\mathbb{E}e^{\lambda X_j} \leq 1 + \sum_{k \geq 2} \frac{\lambda^k \mathbb{E}X_j^2\, b^{k-2}}{k!} \leq 1 + \frac{\lambda^2 \mathbb{E}X_j^2}{2(1 - b\lambda/3)},$$

and $1 + u \leq e^u$ turns this into $\ln \mathbb{E}e^{\lambda X_j} \leq \lambda^2 \mathbb{E}X_j^2/(2(1 - b\lambda/3))$. Independence adds these bounds over j, so $\ln \mathbb{E}e^{\lambda \sum_j X_j} \leq \lambda^2 V/(2(1 - b\lambda/3))$. Markov's inequality applied to $e^{\lambda \sum_j X_j}$ with

$$\lambda = \frac{\sqrt{2t/V}}{1 + (b/3)\sqrt{2t/V}}$$

bounds the probability that $\sum_j X_j > \sqrt{2Vt} + bt/3$ by e^{-t}. The same argument applied to $-X_j$ bounds the lower tail, and the two tails add up to $2e^{-t}$. □

Lemma A.2 (Clopper-Pearson coverage)**.** Let X be binomial with $N \geq 1$ trials and success probability $\pi \in [0, 1]$, let $\alpha \in (0, 1)$, and let $[\ell(X), u(X)]$ be the two-sided Clopper-Pearson interval of Section III-A. Then $\Pr[\pi \notin [\ell(X), u(X)]] \leq \alpha$. Moreover, $\ell(X) \leq X/N \leq u(X)$, and $\ell(X) \geq X/N - \varepsilon$ and $u(X) \leq X/N + \varepsilon$ with $\varepsilon = \sqrt{\ln(2/\alpha)/(2N)}$.

Proof. For $0 < k$, the lower endpoint $\ell(k)$ is the value of π' at which $\Pr_{\pi'}[X \geq k] = \alpha/2$, and the upper tail $\Pr_{\pi'}[X \geq k]$ increases with π'. For a fixed π, the event $\ell(X) > \pi$ is therefore an upper tail $\{X \geq k_0\}$ with $\ell(k_0) > \pi$, so its probability is $\Pr_\pi[X \geq k_0] \leq \Pr_{\ell(k_0)}[X \geq k_0] = \alpha/2$. Symmetrically, the event $u(X) < \pi$ is a lower tail of probability at most $\alpha/2$. The conventions $\ell(0) = 0$ and $u(N) = 1$ cover $\pi = 0$ and $\pi = 1$. This proves the coverage. If $X = k$ and $\pi' = k/N$, both tails through k have probability at least $1/2 > \alpha/2$ because the median of a binomial distribution with integer mean equals its mean, so $\ell(k) \leq k/N \leq u(k)$. Finally, if $k/N - \varepsilon \leq 0$, the bound $\ell(k) \geq k/N - \varepsilon$ holds because $\ell(k) \geq 0$. Otherwise, for $\pi' = k/N - \varepsilon \in (0, 1)$, Hoeffding's

inequality [13] gives $\Pr_{\pi'}[X \geq k] \leq e^{-2N\varepsilon^2} = \alpha/2$, so the increasing tail reaches $\alpha/2$ only at $\pi' \geq k/N - \varepsilon$, which means $\ell(k) \geq k/N - \varepsilon$. Symmetrically, $u(k) \leq 1$ settles the case $k/N + \varepsilon \geq 1$, and otherwise Hoeffding's inequality for the lower tail at $\pi' = k/N + \varepsilon$ gives $u(k) \leq k/N + \varepsilon$. □

D. Proof of Theorem 1 (Region)

Coverage. Under (A2) and (A3), the count $n_{e,z}$ is binomial with N_e trials and success probability $q_e(z)$, for every $e \in \mathcal{S}$ and every $z \in Z_e$, whether z is observed or not. By Lemma A.2, the interval $[\underline{q}_{e,z}, \overline{q}_{e,z}]$ misses $q_e(z)$ with probability at most α_e. The sets Z_e and $\mathcal{S}$ are fixed by (A1), so the union bound over all pairs (e, z) gives

$$\Pr\left[\exists e, z : q_e(z) \notin [\underline{q}_{e,z}, \overline{q}_{e,z}]\right] \leq \sum_{e \in \mathcal{S}} |Z_e|\, \alpha_e = \delta_0.$$

On the complement, q_e satisfies every box constraint of C_e^0, and q_e lies in the simplex, so $q \in C^0$. The union bound requires no independence between the counts.

Nonemptiness. By Lemma A.2, every interval contains the empirical proportion $\hat{q}_e(z)$, and $\hat{q}_e$ lies in the simplex, so $\hat{q}_e \in C_e^0$.

Extrema. Fix e and $a \in \{-1, 0, +1\}^{Z_e}$. Every $r_e \in C_e^0$ can be written as $r_e(z) = \underline{q}_{e,z} + d_z$ with $0 \leq d_z \leq \overline{q}_{e,z} - \underline{q}_{e,z}$ and $\sum_z d_z = \rho$, and every such d gives a member of C_e^0. Then $a^\top r_e = v_0 + \sum_z a_z d_z$. Let $D_t = \sum_{z: a_z = t} d_z$ for $t \in \{-1, 0, +1\}$. The objective equals $v_0 - D_{-1} + D_{+1}$, and the constraints on d allow exactly the triples with $0 \leq D_t \leq W_t$ and $D_{-1} + D_0 + D_{+1} = \rho$, because variables in intervals $[0, c_j]$ reach every sum in $[0, \sum_j c_j]$. To minimize, the mass ρ goes first to the coefficient -1, up to W_{-1}, then to 0, up to W_0, and the rest to $+1$. Any other feasible triple can move mass from a larger coefficient to a smaller one that is not full without increasing the objective. This filling gives $D_{-1} = \min(\rho, W_{-1})$ and $D_{+1} = (\rho - W_{-1} - W_0)_+$, which is the stated minimum. The maximum fills the coefficients in the reverse order. □

E. Proof of Lemma 1 (Propagation)

Let $r \in C^0$. By Theorem 1, every factor value $u_{i,g}(y_i)$ and every mass $S_{i,g}$, evaluated at r, lies in its interval, because each interval is the range of its functional over C_e^0. Interval arithmetic preserves containment. If $\xi \in [\xi_1, \xi_2]$ and $\eta \in [\eta_1, \eta_2]$, then $\xi + \eta \in [\xi_1 + \eta_1, \xi_2 + \eta_2]$, $c\xi$ lies in the interval spanned by $c\xi_1$ and $c\xi_2$, and $\xi\eta$ lies between the smallest and the largest of the four endpoint products, because $\xi\eta$ is bilinear and attains its extrema over a rectangle at a corner. Induction over the expression tree of (3), from the factors to the root, shows that the interval computed for $p(x)$ contains $p_r(x)$, so $L^0(x) \leq p_r(x) \leq U^0(x)$ for every x. □

F. Proof of Theorem 2 (Refinement)

Fix the pilot, and with it A, P, F, the shares of δ and the counts N_e. Let $\Omega_0 = \{q \in C^0\}$, so that $\Pr[\Omega_0^c] \leq \delta_0$ by Theorem 1. Fix $f \in F$ in the form (6), and write $v_{h,i} = v_{h,i}(q)$, $\hat{v}_{h,i} = v_{h,i}(\hat{q})$ and $\Delta_{h,i} = \hat{v}_{h,i} - v_{h,i}$.

Step 1: exact expansion. For every product h,

$$\prod_{i \in I_h} \hat{v}_{h,i} = \prod_{i \in I_h} (v_{h,i} + \Delta_{h,i}) = \sum_{\mathcal{K} \subseteq I_h} \prod_{j \in \mathcal{K}} \Delta_{h,j} \prod_{j \in I_h \setminus \mathcal{K}} v_{h,j}.$$

The term with $\mathcal{K} = \emptyset$ is the product at q. The terms with $|\mathcal{K}| = 1$ are $\Delta_{h,i} \prod_{j \neq i} v_{h,j}$, and $\Delta_{h,i} = \sum_z a_{h,i}(z)(\hat{q}_e(z) - q_e(z))$ with $e = e(h, i)$. Multiplying by $c_{h,f}$ and summing over h and i collects, for every subexperiment e and category z, exactly the weight $g_{e,f}(q, z)$ of (7). Hence

$$f(\hat{q}) - f(q) = Z_f(q) + \mathcal{R}_f, \qquad \mathcal{R}_f = \sum_h c_{h,f} \sum_{|\mathcal{K}| \geq 2} \prod_{j \in \mathcal{K}} \Delta_{h,j} \prod_{j \notin \mathcal{K}} v_{h,j}.$$

Step 2: the remainder. On Ω_0, both $v_{h,j}$ and $\hat{v}_{h,j}$ lie in $[\underline{v}_{h,j}, \overline{v}_{h,j}]$, the first because $q \in C^0$ and the second because $\hat{q} \in C^0$. Hence $|\Delta_{h,j}| \leq r_{h,j}$ and $|v_{h,j}| \leq |\hat{v}_{h,j}| + r_{h,j} = M_{h,j}$, and the triangle inequality gives $|\mathcal{R}_f| \leq R_f$.

Step 3: the linear term. At the true q, $Z_f(q) = \sum_e \sum_{\ell=1}^{N_e} X_{e,\ell}$ with

$$X_{e,\ell} = \frac{g_{e,f}(q, Z_{e,\ell}) - \mathbb{E}_{q_e}\, g_{e,f}(q, Z)}{N_e},$$

where $Z_{e,\ell}$ is the category of shot ℓ of subexperiment e. The weights $g_{e,f}(q, \cdot)$ are fixed given the pilot, so by (A2) the $X_{e,\ell}$ are independent and centered. Each satisfies $|X_{e,\ell}| \leq b_f(q)$, and $\sum_{e,\ell} \mathbb{E} X_{e,\ell}^2 = V_f(q)$. Lemma A.1 gives

$$\Pr\left[|Z_f(q)| > \sqrt{2V_f(q)t_f} + b_f(q)t_f/3\right] \leq 2e^{-t_f}.$$

Call the complement of this event Ω_f.

Step 4: computable envelopes. On Ω_0, for every e and z,

$$|g_{e,f}(q, z) - \hat{g}_{e,f}(z)| \leq \sum_{(h,i): e(h,i) = e} |c_{h,f}|\, |a_{h,i}(z)| \left| \prod_{j \neq i} v_{h,j} - \prod_{j \neq i} \hat{v}_{h,j} \right| \leq d_e$$

The last inequality expands $\prod_{j \neq i}(\hat{v}_{h,j} - \Delta_{h,j}) - \prod_{j \neq i} \hat{v}_{h,j}$ into the terms with at least one factor $\Delta_{h,j}$ and bounds each term in absolute value. The sum of these bounds is $\prod_{j \neq i}(|\hat{v}_{h,j}| + r_{h,j}) - \prod_{j \neq i} |\hat{v}_{h,j}|$. A variance is at most the second moment about any center, and the triangle inequality holds in $L^2(q_e)$, so

$$\sqrt{\mathrm{Var}_{q_e}[g_{e,f}(q, Z)]} \leq \Big(\sum_z q_e(z)(\hat{g}_{e,f}(z) - \mu_{e,f})^2 \Big)^{1/2} + \Big(\sum_z q_e(z)\, d_{e,f}(z$$

where the last step uses $q_e(z) \leq \overline{q}_{e,z}$ on Ω_0. Hence $V_f(q) \leq \overline{V}_f$. The same bound on the weights gives $\max_z g_{e,f}(q, z) \leq \max_z(\hat{g}_{e,f} + d_{e,f})(z)$ and $\min_z g_{e,f}(q, z) \geq \min_z(\hat{g}_{e,f} - d_{e,f})(z)$, so $b_f(q) \leq \overline{b}_f$.

Step 5: the event without the tighter terms. On $\Omega_0 \cap \Omega_f$, Steps 1 to 4 give $|f(\hat{q}) - f(q)| \leq \sqrt{2\overline{V}_f t_f} + \overline{b}_f t_f/3 + R_f = E_f$. Let $\Omega = \Omega_0 \cap \bigcap_{f \in F} \Omega_f$. The union bound over Ω_0^c and the $|A|$ score events and $|P|$ pair events gives

$$\Pr[\Omega^c] \leq \delta_0 + |A| \cdot 2e^{-\ln(2|A|/\delta_s)} + |P| \cdot 2e^{-\ln(2|P|/\delta_p)} = \delta_0 + \delta_s + \delta_p.$$

Step 6: the tighter terms. (i) Factor radii. For a functional $(e, a) \in \mathcal{F}$, $a \cdot (\hat{q}_e - q_e)$ is the sum of the N_e independent centered terms $(a(Z_{e,\ell}) - a \cdot q_e)/N_e$, each bounded by $(\max a -$

$\min a)/N_e$, with total variance $\mathrm{Var}_{q_e}[a(Z)]/N_e$. Lemma A.1 with $t_{\mathcal{F}}$ shows that the event

$$\Omega_a = \Big\{ |a\cdot(\hat{q}_e - q_e)| \le \sqrt{2\,\mathrm{Var}_{q_e}[a(Z)]\,t_{\mathcal{F}}/N_e} + (\max a - \min a)\,t_{\mathcal{F}}/(3N_e) \Big\}$$

fails with probability at most $2e^{-t_{\mathcal{F}}}$, and the union bound over the $|\mathcal{F}|$ distinct functionals gives $\Pr[\bigcup_a \Omega_a^c] \le \delta_{\mathcal{F}}$. On Ω_0, $\mathrm{Var}_{q_e}[a(Z)] \le \sum_z q_e(z)(a_z - a\cdot\hat{q}_e)^2 \le \bar{\sigma}_a^2$, because a variance is at most the second moment about $a\cdot\hat{q}_e$ and $q_e \in C_e^0$. Hence, on $\Omega_0 \cap \bigcap_a \Omega_a$, every functional satisfies $|\Delta_{h,i}| \le \rho_a$. Steps 2 and 4 use the radius $r_{h,i}$ only through the two facts $|\Delta_{h,i}| \le r_{h,i}$ and $|v_{h,i}| \le |\hat{v}_{h,i}| + r_{h,i}$ at the true q, and both facts hold with $\min\{r_{h,i}, \rho_a\}$ in place of $r_{h,i}$ on this event. *(ii) Envelopes.* Step 4 uses $\sum_z q_e(z)\phi(z)^2 \le \sum_z \overline{q}_{e,z}\phi(z)^2$ for $\phi = \hat{g}_{e,f} - \mu_{e,f}$ and $\phi = d_{e,f}$. On Ω_0, $q_e \in C_e^0$, so $\sum_z q_e(z)\phi(z)^2 \le \sup_{r\in C_e^0}\sum_z r(z)\phi(z)^2$ as well. This supremum maximizes a linear function over C_e^0, and the greedy fill of the proof of Theorem 1, applied to the values $\phi(z)^2$ in decreasing order, computes it exactly. *(iii) Bilinear remainder.* Suppose that every product of f has two factors, $v_{h,0}$ on a subexperiment in $\mathcal{E}_0$ and $v_{h,1}$ on a subexperiment in $\mathcal{E}_1$, with $\mathcal{E}_0 \cap \mathcal{E}_1 = \emptyset$. Then $\mathcal{R}_f = \sum_h c_{h,f}\Delta_{h,0}\Delta_{h,1}$ by Step 1. Condition on all shots of the subexperiments in $\mathcal{E}_1$. The $\Delta_{h,1}$ are then fixed, and

$$\mathcal{R}_f = \sum_{e\in\mathcal{E}_0}\sum_{\ell=1}^{N_e} \frac{w_e(Z_{e,\ell}) - \mathbb{E}_{q_e} w_e(Z)}{N_e}, \qquad w_e(z) = \sum_{h:e(h,0)=e} c_{h,f}\,a_{h,0}(z)\,\Delta_{h,1}.$$

By (A2), the shots of $\mathcal{E}_0$ are independent of the conditioning shots, so Lemma A.1 applies conditionally. With probability at least $1 - 2e^{-t_f}$, conditionally and hence unconditionally,

$$|\mathcal{R}_f| \le \sqrt{2t_f \textstyle\sum_e \mathrm{Var}_{q_e}[w_e(Z)]/N_e} + t_f \max_e(\max_z w_e - \min_z w_e)/(3N_e).$$

On $\Omega_0 \cap \bigcap_a \Omega_a$, $|\Delta_{h,1}| \le \min\{r_{h,1}, \rho_{a_{h,1}}\}$, where $a_{h,1}$ is the functional of the factor $v_{h,1}$, so $|w_e(z)| \le G_e(z) = \sum_{h:e(h,0)=e} |c_{h,f}|\,|a_{h,0}(z)|\min\{r_{h,1}, \rho_{a_{h,1}}\}$. Hence $\mathrm{Var}_{q_e}[w_e(Z)] \le \sum_z q_e(z)G_e(z)^2 \le \sup_{r\in C_e^0}\sum_z r(z)G_e(z)^2$ and $\max_z w_e - \min_z w_e \le 2\max_z G_e(z)$, and the right-hand side of the last display is at most a computable bound $\overline{B}_f$. On the intersection of all these events, $|\mathcal{R}_f| \le \min\{R_f, \overline{B}_f\}$.

Step 7: the level. With the tighter terms, each f carries the linear event of Step 3 and the bilinear event of (iii), each failing with probability at most $2e^{-t_f}$, and $t_f = \ln(4|A|/\delta_s)$ for scores and $t_f = \ln(4|P|/\delta_p)$ for pairs. The union bound gives

$$\Pr[\Omega^c] \le \delta_0 + \delta_{\mathcal{F}} + |A|\cdot 4e^{-t_s} + |P|\cdot 4e^{-t_p} = \delta_0 + \delta_{\mathcal{F}} + \delta_s + \delta_p = \delta.$$

On Ω, $|f(\hat{q}) - f(q)| \le E_f$ for every $f \in F$, with E_f computed from the tighter terms, and $q \in C^0$. Averaging over the pilot completes the proof. □

G. Proof of Theorem 3 (Soundness)

By Lemma 1 and Theorem 2, on Ω every string satisfies $L(x) \le p(x) \le U(x)$. For $x \notin A$ this is Lemma 1 at $r = q$. For $x \in A$, the score radius gives $\hat{p}(x) - E_x \le p(x) \le \hat{p}(x) + E_x$, and the maximum and minimum in (9) keep both inclusions. Every pair $\{x, y\} \in P$ with $x \succ y$ satisfies $p(x) - p(y) \ge \hat{p}(x) - \hat{p}(y) - E_{xy} > 0$. In particular, no refined interval is empty on Ω.

Threshold query. $L(x) > \tau$ implies $p(x) \ge L(x) > \tau$, and $U(x) < \tau$ implies $p(x) < \tau$.

Top-k query, IN. Every $y \in \mathrm{Below}(x)$ has $p(y) < p(x)$, either because $p(y) \le U(y) < L(x) \le p(x)$ or because $x \succ y$. Hence $\{y \ne x : p(y) \ge p(x)\} \subseteq \{y \ne x\} \setminus \mathrm{Below}(x)$, which has $2^m - 1 - |\mathrm{Below}(x)| \le k - 1$ elements when x is labelled IN. By the characterization in Section II-C, x is a sure member.

Top-k query, OUT. Every $y \in \mathrm{Above}(x)$ has $p(y) > p(x)$ by the symmetric argument. If $|\mathrm{Above}(x)| \ge k$, at least k strings have a strictly larger value, so x is a sure non-member. A sure member has at most $k - 1$ strings of larger or equal value, so it cannot be a sure non-member, and the two tests never hold together on Ω.

The argument is deterministic given Ω and applies to every string, so $\Pr[\text{every IN and OUT label is correct}] \ge \Pr[\Omega] \ge 1 - \delta$. □

H. Proof of Lemma 2 (Branch bound)

Let x complete the branch B. For each group g and subcircuit i, if the block x_i is fixed completely by B, the interval that B uses for the factor is the interval of x. Otherwise the interval of B is the hull over all block values that agree with the fixed bits, and x_i is one of these values, so it contains the interval of x. The masses and the constant term do not depend on x. Every interval operation is monotone under containment. If $[\xi] \subseteq [\xi']$ and $[\eta] \subseteq [\eta']$, the sum, the product with a coefficient and the product of $[\xi]$ and $[\eta]$ lie inside the corresponding results for $[\xi']$ and $[\eta']$. Induction over the expression of (3) shows that the interval of $p(x)$ lies inside the interval computed for B, so $U^0(x) \le \overline{U}(B)$. The relative slack of the implementation only enlarges $\overline{U}(B)$. □

I. Proof of Theorem 4 (Search equals enumeration)

Collection. Algorithm 1 returns exactly $\{x : U^0(x) \ge \theta\}$. Every string it returns passes the final filter. Conversely, let $U^0(x) \ge \theta$. Every branch B on the path from the empty prefix to x contains x, so $\overline{U}(B) \ge U^0(x) \ge \theta$ by Lemma 2, and no branch on this path is discarded. Hence x reaches the last level and passes the filter.

Beam bound. The beam pass computes L^0 for a subset of the strings, so the k-th largest L^0 it sees is at most $L^0_{(k)}$, and $\lambda \le L^0_{(k)}$. Every string with $L^0(x) \ge L^0_{(k)}$ has $U^0(x) \ge L^0(x) \ge \lambda \ge \theta$ and is in $\mathcal{C}$, so the k largest values of L^0 over all strings are among the values of $\mathcal{C}$, and $L^0_{(k)}$ is read exactly. By the definition of θ_k, every $x \in A$ has $U^0(x) \ge U(x) \ge \theta_k$, so $A \subseteq \mathcal{C}_k$.

Top-k labels. Let W be the set of k strings with the largest L^0. We compare the label of every string x with the label that enumeration assigns. (a) $x \notin \mathcal{C}_k$. Then $x \notin A$, so $U(x) = U^0(x) < \theta_k \le L^0_{(k)}$. Every $w \in W$ satisfies $L(w) \ge L^0(w) \ge L^0_{(k)} > U(x)$, and $w \ne x$ because $U^0(w) \ge L^0(w) > U^0(x)$. Hence $W \subseteq \mathrm{Above}(x)$, enumeration labels x OUT, and so does the search. (b) $x \in \mathcal{C}_k$. Recall that both

strings of every pair in P belong to A (Section III-C). Every string $y \notin \mathcal{C}_k$ satisfies $L(y) \le U(y) = U^0(y) < \theta_k \le U(x)$, so $y \notin \mathrm{Above}(x)$, and the pair tests involve only strings of $A \subseteq \mathcal{C}_k$. Hence $|\mathrm{Above}(x)|$ is the same whether it is counted inside $\mathcal{C}_k$ or over all strings, and the OUT test agrees. For the IN test, suppose first that $L(x) \ge \theta_k$. Every $y \notin \mathcal{C}_k$ then has $U(y) < \theta_k \le L(x)$, so $y \in \mathrm{Below}(x)$, and the number of competitors counted inside $\mathcal{C}_k$ equals the number over all strings. Suppose instead that $L(x) < \theta_k$. Then $x \notin A$, since every string of A has $L \ge \theta_k$ by the definition of θ_k, so x has no pair test and $\mathrm{Below}(x) = \{y : U(y) < L(x)\}$. Every $w \in W \setminus \{x\}$ has $U(w) \ge L^0(w) \ge L^0_{(k)} \ge \theta_k > L(x)$, so $w \notin \mathrm{Below}(x)$. If $x \notin W$, these are k competitors. If $x \in W$, then $L(x) \ge L^0(x) \ge L^0_{(k)} \ge \theta_k$, which contradicts $L(x) < \theta_k$. So enumeration leaves at least k competitors and does not label x IN, and the search does not either. (c) The empty-interval case depends only on the strings of A, which are in $\mathcal{C}_k$, so both procedures detect it alike.

Threshold labels. A string outside $\mathcal{C}_\tau$ is not in A and has $U(x) = U^0(x) < \tau$, so both procedures label it OUT. Inside $\mathcal{C}_\tau$ the threshold rule depends on the string alone.

Cost. The walk has m levels. At each level it computes the branch bound of the two children of every kept branch, each from the hull pyramids of the blocks at $O(|\mathcal{G}|\, s)$ interval operations, and it keeps only the current level. □

J. Proof of Theorem 5 (Appended shots)

Let $\mathcal{D}_{t-1}$ be the data $D_0, \ldots, D_{t-1}$. In round t, the batch sizes, the selected strings, the pairs and the error shares are functions of $\mathcal{D}_{t-1}$. Given $\mathcal{D}_{t-1}$, the batch D_t consists of new shots that are independent of $\mathcal{D}_{t-1}$ and of each other by (A2), with sizes that are fixed given $\mathcal{D}_{t-1}$. So (A1) to (A3) hold for D_t conditionally on $\mathcal{D}_{t-1}$, with $\mathcal{D}_{t-1}$ in the role of the pilot. Theorem 2 at level δ_t gives $\Pr[\Omega_t^c \mid \mathcal{D}_{t-1}] \le \delta_t$, and averaging gives $\Pr[\Omega_t^c] \le \delta_t$. Define the rounds after the stopping round by any fixed continuation rule. The events Ω_t are then defined for all $t \ge 1$, whatever the stopping rule, and

$$\Pr\Big[\bigcup_{t\ge1} \Omega_t^c\Big] \le \sum_{t\ge1} \delta\, 2^{-t} = \delta.$$

On $\bigcap_t \Omega_t$, every round satisfies $L_r(x) \le p(x) \le U_r(x)$ for every string, so $L_t^*(x) = \max_{r\le t} L_r(x) \le p(x) \le \min_{r\le t} U_r(x) = U_t^*(x)$, and every pair shown in any round is correct. The proof of Theorem 3 uses only these two facts, so every label issued in every round is correct. As t grows, L_t^* can only rise, U_t^* can only fall, and the set of pairs can only grow. Hence $\mathrm{Below}(x)$ and $\mathrm{Above}(x)$ can only grow, and an IN or OUT test that holds in round t holds in every later round. On $\bigcap_t \Omega_t$, the two tests never hold together and no running interval is empty, by the proof of Theorem 3, so the IN or OUT label itself is kept. A threshold label is kept for the same reason. Both statements hold on $\bigcap_t \Omega_t$, an event of probability at least $1 - \delta$. □

K. Proof of Theorem 6 (Eventual decision)

Width of the bounds. Every factor and mass is $a^\top r_e$ with $a \in \{-1, 0, +1\}^{Z_e}$ and r_e in the simplex, so its value lies in $[-1, 1]$, and so does its range over C_e^0. That range has width at most $\sum_z |a_z|(\overline{q}_{e,z} - \underline{q}_{e,z}) \le W_e$, because C_e^0 is contained in the box. For intervals $[\xi] \subseteq [-1, 1]$ and $[\eta] \subseteq [-1, 1]$ of widths w_ξ and w_η, the interval product has width at most $w_\xi \max|\eta| + w_\eta \max|\xi| \le w_\xi + w_\eta$ and again lies in $[-1, 1]$. By induction, a product of at most $\bar{s}$ factor intervals has width at most $\bar{s} \max_e W_e$. The interval of $T(x)$ therefore has width at most $\sum_g |c_g|\, \bar{s} \max_e W_e$, and the constant term of (3) has width at most $2^{-m} \sum_g |c_g|\, \bar{s} \max_e W_e$. Adding the two gives $U^0(x) - L^0(x) \le \Delta$, and the refined bounds (9) are never wider than $[L^0(x), U^0(x)]$. By Lemma A.2, every Clopper-Pearson interval has width at most $2\sqrt{\ln(2/\alpha_e)/(2N_e)}$, which gives the stated bound on W_e.

Decisions. On Ω, $L(x) \le p(x) \le U(x)$ and $U(x) - L(x) \le \Delta$, so $L(x) \ge p(x) - \Delta$ and $U(x) \le p(x) + \Delta$. For the threshold query, $p(x) - \tau > \Delta$ gives $L(x) > \tau$, so x is IN, and $\tau - p(x) > \Delta$ gives $U(x) < \tau$, so x is OUT. For the top-k query with $k < 2^m$ and $\gamma > 0$, the top-k set $\mathcal{T}$ is unique. If $\Delta < \gamma/2$, every $x \in \mathcal{T}$ and every $y \notin \mathcal{T}$ satisfy $L(x) \ge p_{(k)} - \Delta > p_{(k+1)} + \Delta \ge U(y)$. Hence every $y \notin \mathcal{T}$ lies in $\mathrm{Below}(x)$, at most $k - 1$ competitors remain, and x is IN. Every $y \notin \mathcal{T}$ has the k strings of $\mathcal{T}$ in $\mathrm{Above}(y)$, so y is OUT.

Rounds. In round t, step 3 of the protocol splits $\delta_t = \delta 2^{-t}$ in fixed proportions, so the region uses a fixed fraction $c \in (0, 1]$ of δ_t, so $\ln(2|\mathcal{S}||Z_e|/(c\delta_t)) = \ln(2|\mathcal{S}||Z_e|/(c\delta)) + t \ln 2$, while $N_{e,t} \ge \beta 2^{t-1} n_1$. The bound on W_e for the batch of round t is therefore of order $\sqrt{t/2^t}$, and $\Delta_t \to 0$. On $\bigcap_t \Omega_t$, the running bounds satisfy $L_t^*(x) \ge L_t(x) \ge p(x) - \Delta_t$ and $U_t^*(x) \le p(x) + \Delta_t$, and the running pairs contain the pairs of round t. The argument above therefore applies to the running bounds as soon as $\Delta_t < |p(x) - \tau|$ or $\Delta_t < \gamma/2$, which happens after finitely many rounds. □

References

[1] T. Peng, A. W. Harrow, M. Ozols, and X. Wu, "Simulating large quantum circuits on a small quantum computer," *Physical Review Letters*, vol. 125, no. 15, p. 150504, 2020.

[2] K. Mitarai and K. Fujii, "Constructing a virtual two-qubit gate by sampling single-qubit operations," *New Journal of Physics*, vol. 23, no. 2, p. 023021, 2021.

[3] W. Tang, T. Tomesh, M. Suchara, J. Larson, and M. Martonosi, "CutQC: Using small quantum computers for large quantum circuit evaluations," in *Proc. ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2021, pp. 473–486.

[4] J. Nivala and FiQCI, "QCut: Circuit knitting library, version 2.4.0," https://github.com/FiQCI/QCut/releases/tag/v2.4.0, Nov. 2026, software release, November 4, 2026.

[5] N. M. P. Neumann, C. M. R. Rocha, J. Verbree, and M. van Vliet, "Probability distribution reconstruction using circuit cutting applied to a variational classifier," arXiv:2510.03077, 2025.

[6] M. A. Soliman, I. F. Ilyas, and K. C.-C. Chang, "Top-k query processing in uncertain databases," in *Proc. IEEE ICDE*, 2007, pp. 896–905.

[7] C. Ré, N. Dalvi, and D. Suciu, "Efficient top-k query evaluation on probabilistic data," in *Proc. IEEE ICDE*, 2007, pp. 886–895.

[8] M. Hua, J. Pei, W. Zhang, and X. Lin, "Ranking queries on uncertain data: A probabilistic threshold approach," in *Proc. ACM SIGMOD*, 2008, pp. 673–686.

[9] M. Dylla, I. Miliaraki, and M. Theobald, "Top-k query processing in probabilistic databases with non-materialized views," in *Proc. IEEE ICDE*, 2013, pp. 122–133.

[10] S. Boucheron, G. Lugosi, and P. Massart, *Concentration Inequalities: A Nonasymptotic Theory of Independence*. Oxford University Press, 2013.

[11] J. Shen and H. Zhong, "Ranking queries with confidence guarantees over quantum circuit-cutting reconstructions (extended version, code and data)," Zenodo, https://doi.org/10.5281/zenodo.24120178; code at https://github.com/Mercury0828/rankcert-icde2027, 2026.

[12] C. J. Clopper and E. S. Pearson, "The use of confidence or fiducial limits illustrated in the case of the binomial," *Biometrika*, vol. 26, no. 4, pp. 404–413, 1934.

[13] W. Hoeffding, "Probability inequalities for sums of bounded random variables," *Journal of the American Statistical Association*, vol. 58, no. 301, pp. 13–30, 1963.

[14] P.-H. Chen, B.-H. Chen, D.-W. Chiou, and J.-H. R. Jiang, "ShotQC: Reducing sampling overhead in quantum circuit cutting," arXiv:2412.17704, 2024.

[15] S. A. Cuccaro, T. G. Draper, S. A. Kutin, and D. P. Moulton, "A new quantum ripple-carry addition circuit," 2004.

[16] P. Massart, "The tight constant in the Dvoretzky–Kiefer–Wolfowitz inequality," *The Annals of Probability*, vol. 18, no. 3, pp. 1269–1283, 1990.

[17] W. Tang and M. Martonosi, "TensorQC: Towards scalable distributed quantum computing via tensor networks," arXiv:2502.03445, 2025.

[18] A. Daskin, "Certifying quantum optimization and circuit cutting by using quantum–classical moment duality," arXiv:2606.23727, 2026.

[19] I. Trummer and C. Koch, "Multiple query optimization on the D-Wave 2X adiabatic quantum computer," *Proc. VLDB Endowment*, vol. 9, no. 9, pp. 648–659, 2016.

[20] H. Zhong, J. Shen, L. Fan, X. Zhang, H. Wang, M. Pan, and Z. Han, "UNIQ: Communication-efficient distributed quantum computing via unified nonlinear integer programming," *IEEE Journal on Selected Areas in Communications*, 2026.

[21] R. Fagin, A. Lotem, and M. Naor, "Optimal aggregation algorithms for middleware," *Journal of Computer and System Sciences*, vol. 66, no. 4, pp. 614–656, 2003.

[22] I. F. Ilyas, G. Beskales, and M. A. Soliman, "A survey of top-k query processing techniques in relational database systems," *ACM Computing Surveys*, vol. 40, no. 4, pp. 11:1–11:58, 2008.

[23] J. M. Hellerstein, P. J. Haas, and H. J. Wang, "Online aggregation," in *Proc. ACM SIGMOD*, 1997, pp. 171–182.

[24] S. Agarwal, H. Milner, A. Kleiner, A. Talwalkar, M. Jordan, S. Madden, B. Mozafari, and I. Stoica, "Knowing when you're wrong: Building fast and reliable approximate query processing systems," in *Proc. ACM SIGMOD*, 2014, pp. 481–492.

[25] S. Kalyanakrishnan, A. Tewari, P. Auer, and P. Stone, "PAC subset selection in stochastic multi-armed bandits," in *Proc. International Conference on Machine Learning (ICML)*, 2012, pp. 655–662.

[26] C. Réda, E. Kaufmann, and A. Delahaye-Duriez, "Top-m identification for linear bandits," in *Proc. International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. PMLR, vol. 130, 2021, pp. 1108–1116.

[27] W. Xu, H. Mao, H. Shao, W. Liang, J. Peng, W. Huang, Z. Xu, P. Zhou, and J. X. Yu, "An adaptive sampling algorithm for the top-K group betweenness centrality," in *Proc. IEEE ICDE*, 2025, pp. 170–182.

[28] S. R. Howard, A. Ramdas, J. McAuliffe, and J. Sekhon, "Time-uniform, nonparametric, nonasymptotic confidence sequences," *The Annals of Statistics*, vol. 49, no. 2, pp. 1055–1080, 2021.

[29] Y. Sui and Y. Tang, "Certify the decision, not every score: Exact top-k selection from few evaluations," arXiv:2610.14996, 2026.

[30] M. Abo Khamis, H. Q. Ngo, and A. Rudra, "FAQ: Questions asked frequently," in *Proc. ACM PODS*, 2016, pp. 13–28.

[31] ——, "FAQ: Questions asked frequently," arXiv:1504.04044v7, 2015, version 7, December 23, 2023.

[32] D. Olteanu and M. Schleich, "Factorized databases," *ACM SIGMOD Record*, vol. 45, no. 2, pp. 5–16, 2016.

[33] N. Tziavelis, D. Ajwani, W. Gatterbauer, M. Riedewald, and X. Yang, "Optimal algorithms for ranked enumeration of answers to full conjunctive queries," *Proc. VLDB Endowment*, vol. 13, no. 9, pp. 1582–1597, 2020.